Guía práctica de Linux Networking

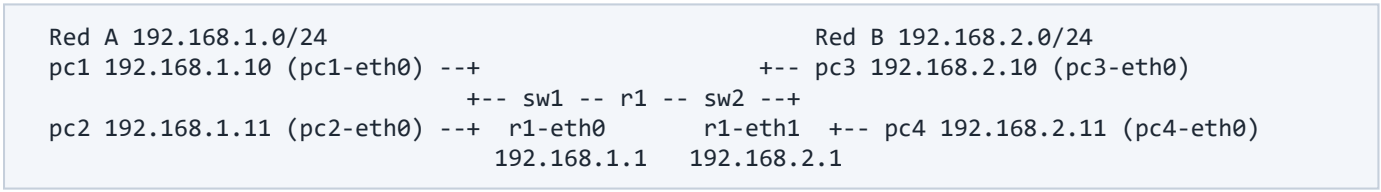
Ubuntu 24.04.4 Desktop: registro de capturas, paso a paso

Máquina virtual	Ubunto_Networking (VirtualBox 7.0.20, host Windows 11)
Sistema	Ubuntu 24.04.4 LTS (Desktop, GNOME) · kernel 7.0.0-42-generic
Recursos	4096 MB de RAM · 2 procesadores · disco de 25 GB dinámico · 128 MB de vídeo
Red de la VM	Adaptador puente (Wi-Fi Intel AX211) · IP 192.168.1.107 · acceso por SSH desde Windows PowerShell
Fecha	03/11/2026
Total de capturas	74 (carpeta `capturas\`)

Cada captura tiene un título, la sección de la guía o del informe a la que corresponde, qué se hizo y, en amarillo, los valores anotados para las tablas de Resultados. Las capturas de las secciones 1 a 3 son de la pantalla de la VM (consola GNOME). Desde la sección 4 son de la ventana de Windows PowerShell conectada por SSH, como recomienda la sección 7 de la guía. Las figuras que pide la guía usan su número (`figNN-ubuntu-…`).

> **Notas:** (1) La red Wi-Fi real del host también es 192.168.1.0/24, igual que la red A de la topología. No interfiere, porque los namespaces están aislados de la red del host. (2) Los scripts `armar_topologia.sh` y `limpiar.sh` se escribieron según la topología que describe la guía (están en `scripts\` y en `comandos_y_scripts.txt`). (3) Para automatizar las capturas se usó una clave SSH y un permiso sudo temporal, que se borró al final.

Topología



Resumen de resultados (columna Ubuntu)

Tabla 4 — Máquina	
Versión	Ubuntu 24.04.4 LTS (Desktop)
Kernel	7.0.0-42-generic
RAM / procesadores / disco	4 GB (3911 MB visibles) / 2 / 25 GB

Tabla 7 — Direcciones MAC	
pc1-eth0	6a:88:12:c1:c7:f3
r1-eth0	e2:7b:f3:01:1d:47
r1-eth1	be:c3:38:bf:45:92
pc3-eth0	22:37:f2:5f:54:d0

Tabla 8 — Exp. 1	**Rep 1**	**Rep 2**	**Rep 3**	**Rep 4**	**Rep 5**	**Promedio**
Tiempo real del armado (s)	2,318	2,435	2,227	2,214	2,313	**2,301**

Tiempo real de la limpieza (s)	0,205	0,241	0,203	0,205	0,202	**0,211**
used antes (MB)	1066	1067	1068	1066	1066	**1066,6**
used después (MB)	1068	1069	1069	1067	1068	**1068,2** (+1,6 MB)

Tabla 9 — Exp. 2	
pc1 → pc2 (misma red)	0 % de pérdida · rtt 0,053/0,302/1,811 ms
pc1 → pc3 sin rutas	«ping: connect: La red es inaccesible»
Con rutas, ip_forward = 0	100 % de pérdida
Con ip_forward = 1 (5 × 20 pings)	0 % de pérdida · rtt avg: 0,154 / 0,244 / 0,479 / 0,917 / 0,145 ms → **promedio 0,388 ms** · min 0,048 ms · max 6,483 ms
TTL / saltos del traceroute	ttl = 63 · 2 saltos (192.168.1.1 → 192.168.2.10)

Tabla 10 — tcpdump en r1-eth1 (ping -c 3)	
Paquetes ARP / ICMP / total	2 / 6 / 8
MAC origen → destino (echo request)	be:c3:38:bf:45:92 (r1-eth1) → 22:37:f2:5f:54:d0 (pc3-eth0)
En r1-eth0 (opcional)	6a:88:12:c1:c7:f3 (pc1-eth0) → e2:7b:f3:01:1d:47 (r1-eth0); también 8 paquetes

Tabla 11 — Firewall ICMP	
pc3 → pc1 (red B inicia)	100 % de pérdida
pc1 → pc3 (red A inicia)	0 % de pérdida
Contador de la regla	4 paquetes (336 bytes)

Tabla 12 — nmap	**Abiertos**	**Filtrados**	**Tiempo**	**Descartados**
Sin firewall	80, 81, 82	—	13,19 s	—
Con firewall (81-82)	80	81, 82	14,39 s	4 paquetes (176 bytes)

Tabla 13 — iperf3 (Gbits/s, receiver)	**R1**	**R2**	**R3**	**R4**	**R5**	**Promedio**
Sin reglas	7,88	7,77	7,25	7,91	8,40	**7,84**
10 reglas	6,73	6,49	7,74	7,47	7,90	**7,27**
100 reglas	6,21	6,66	4,74	6,26	6,99	**6,17**

Tabla 14 — Estado de Ubuntu y configuración persistente	
Kernel	7.0.0-42-generic
Paquetes instalados	1522
Espacio usado	9,6 GB de 25 GB (42 %)
RAM usada	1072 MB de 3911 MB
Archivos de red	/etc/netplan/01-network-manager-all.yaml (renderer NetworkManager), 50-cloud-init.yaml
Servicios	NetworkManager active · systemd-networkd inactive · systemd-resolved active
DNS	127.0.0.53 (stub de systemd-resolved), search lan
Firewall	ufw instalado, inactivo · nft ruleset vacío
sudo	Sí, /usr/bin/sudo

Parte B: herramienta / tiempo	nmcli · real 2,230 s
Dónde se guarda	/etc/netplan/90-NM-<uuid>.yaml (NetworkManager con backend Netplan; system-connections queda vacío)
¿IP mantenida tras reiniciar?	Sí: dummy0 10.10.10.1/24

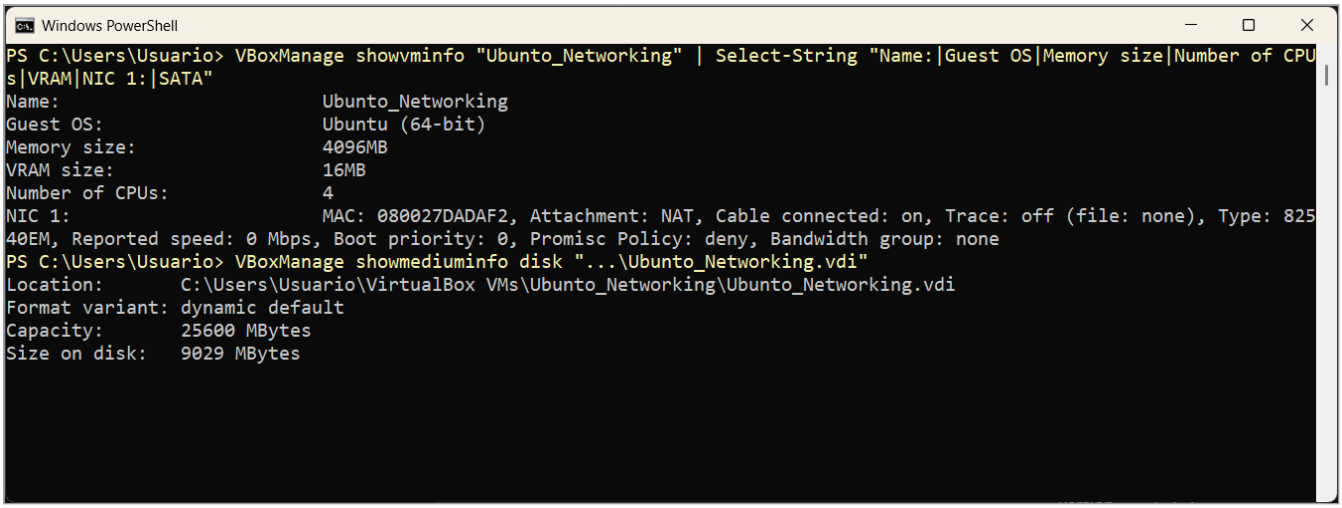
Tabla 15 — Problema	**Causa**	**Solución**	**Minutos**
Durante apt install apareció la ventana «Actualización de software» y tapó la pregunta de iperf3	update-notifier de Ubuntu (128 actualizaciones pendientes)	Volver a la Terminal (Alt+Tab) y responder «No»	≈1 (16:40–16:41)
Después de `nmcli connection delete dummy0`, `nmcli connection show` todavía la mostraba	NetworkManager recargando la configuración de Netplan	Se verificó que el YAML 90-NM-* se borró de /etc/netplan	<1

Capturas

Captura 1 · 1. Crear la máquina virtual

Configuración original de la VM

Con VBoxManage showvminfo se revisa la VM «Ubunto_Networking»: Ubuntu (64-bit), 4096 MB de RAM, 4 CPUs, 16 MB de vídeo, red NAT y disco virtual dinámico de 25600 MB (25 GB).



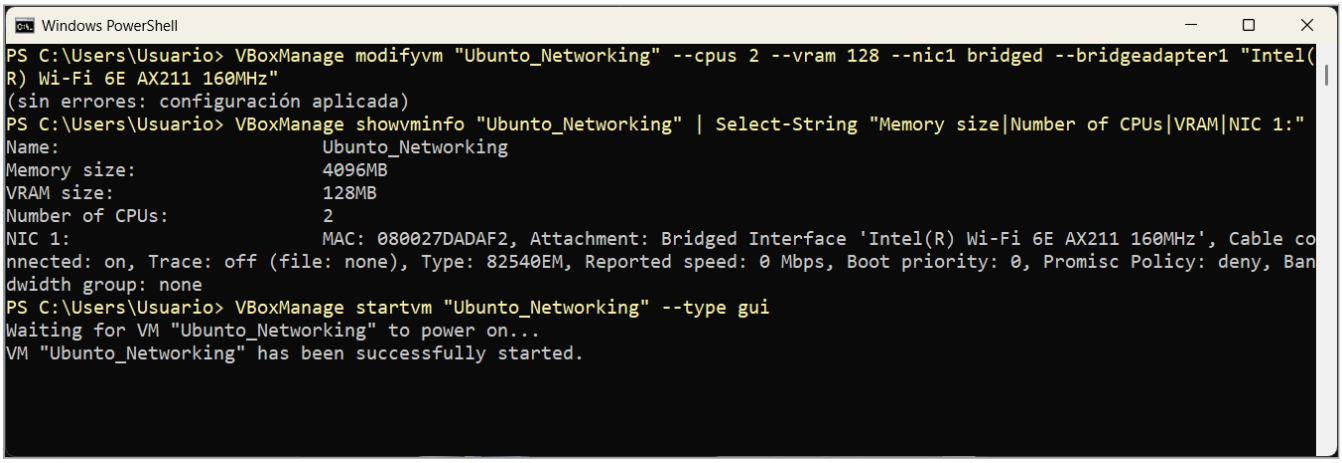
> **Anotar:** Antes del ajuste: 4 CPUs y NAT, distinto de lo que pide la guía.

`capturas\01-ubuntu-vm-config-original.png`

Captura 2 · 1. Crear la máquina virtual

Ajuste de la VM según la guía y arranque

Con modifyvm se igualan los recursos a los de la VM de Debian: 2 procesadores, 128 MB de vídeo y Adaptador 1 en modo puente sobre la Wi-Fi del equipo. Después se inicia la VM con startvm.



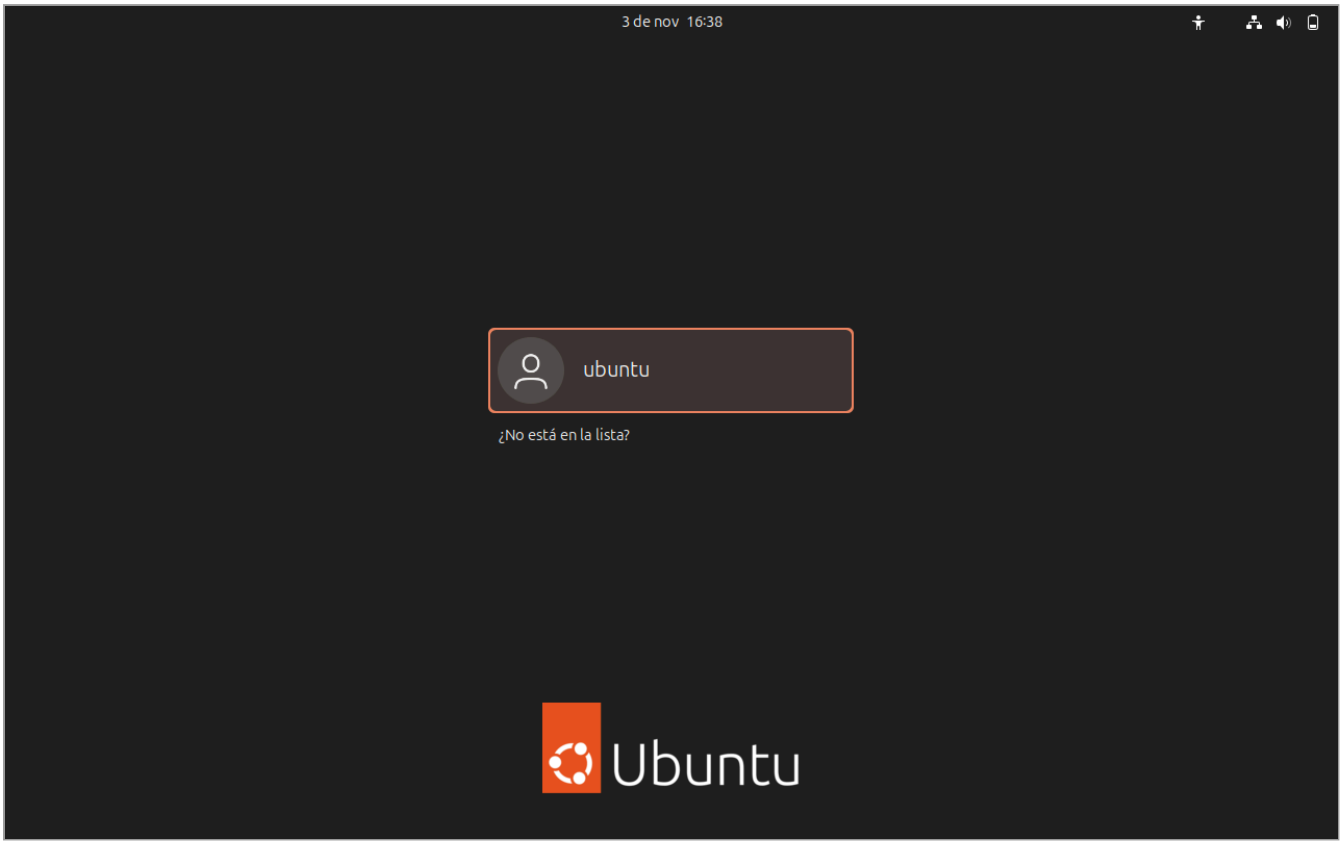
> **Anotar:** Tabla 4: 4096 MB de RAM, 2 CPUs, disco de 25 GB, adaptador puente.

`capturas\02-ubuntu-vm-config-ajustada.png`

Captura 3 · 1-2. VM iniciada

Pantalla de inicio de sesión de Ubuntu

Ubuntu 24.04.4 Desktop ya instalado arranca en la pantalla de inicio de sesión de GNOME con el usuario «ubuntu». Captura tomada con VBoxManage controlvm screenshotpng.

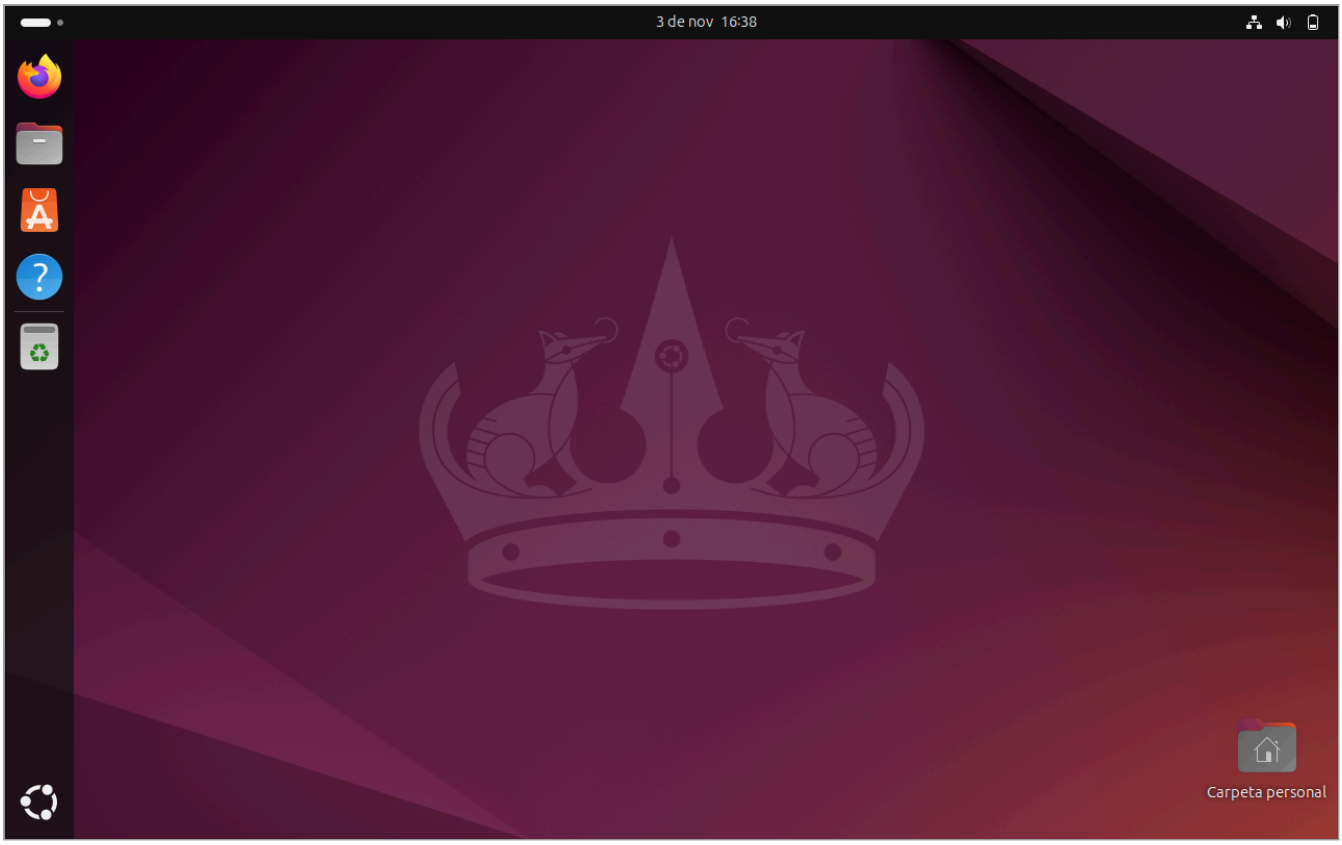


`capturas\03-ubuntu-vm-arrancada.png`

Captura 4 · 2. Instalar Ubuntu (paso 8)

Escritorio de GNOME después del inicio de sesión

Se inicia sesión con el usuario ubuntu y se llega al escritorio. Desde acá se abre la Terminal con Ctrl+Alt+T.

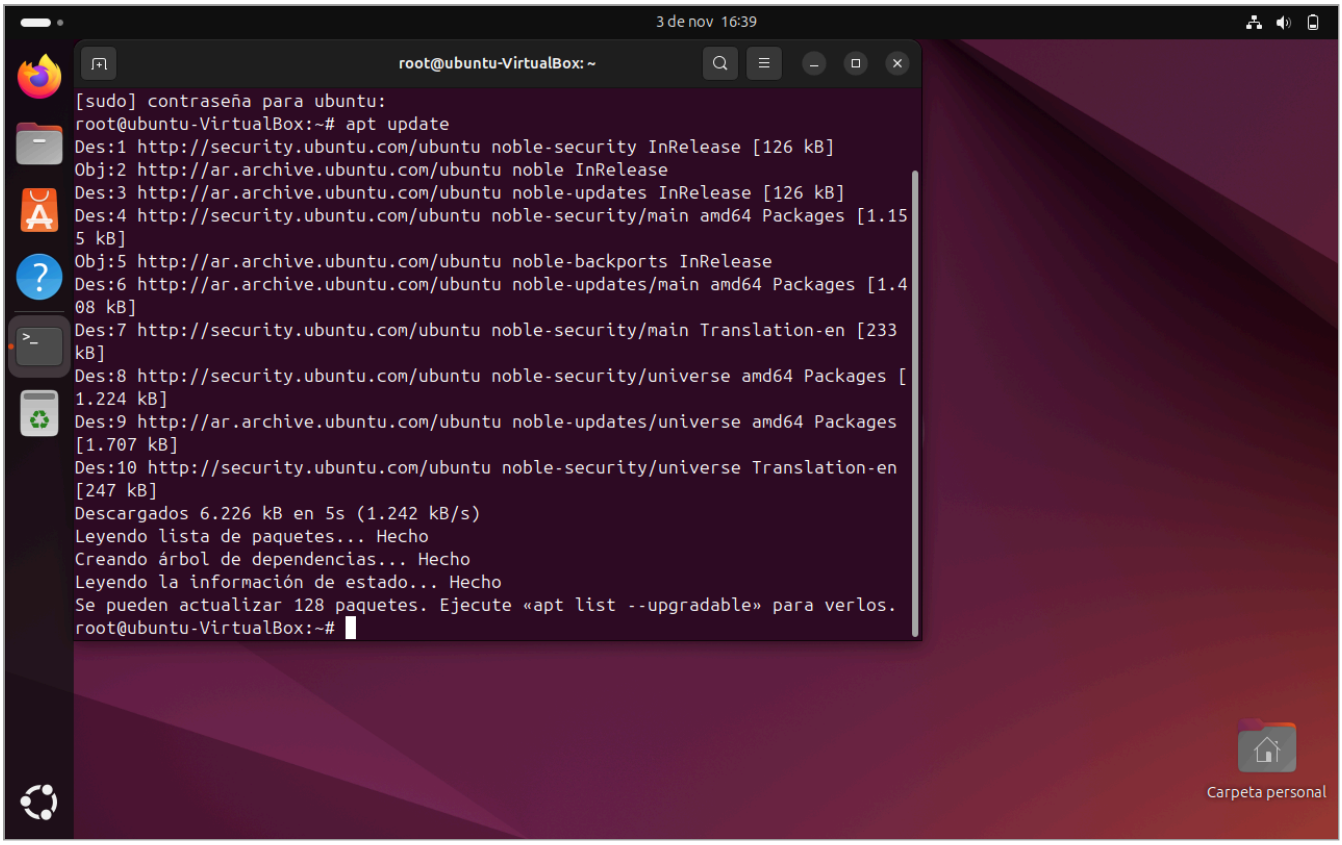


`capturas\04-ubuntu-login.png`

Captura 5 · 3. Preparar la máquina

sudo -i y apt update

En la Terminal de GNOME se pasa a root con sudo -i (la contraseña no se ve) y se actualiza la lista de paquetes con apt update.



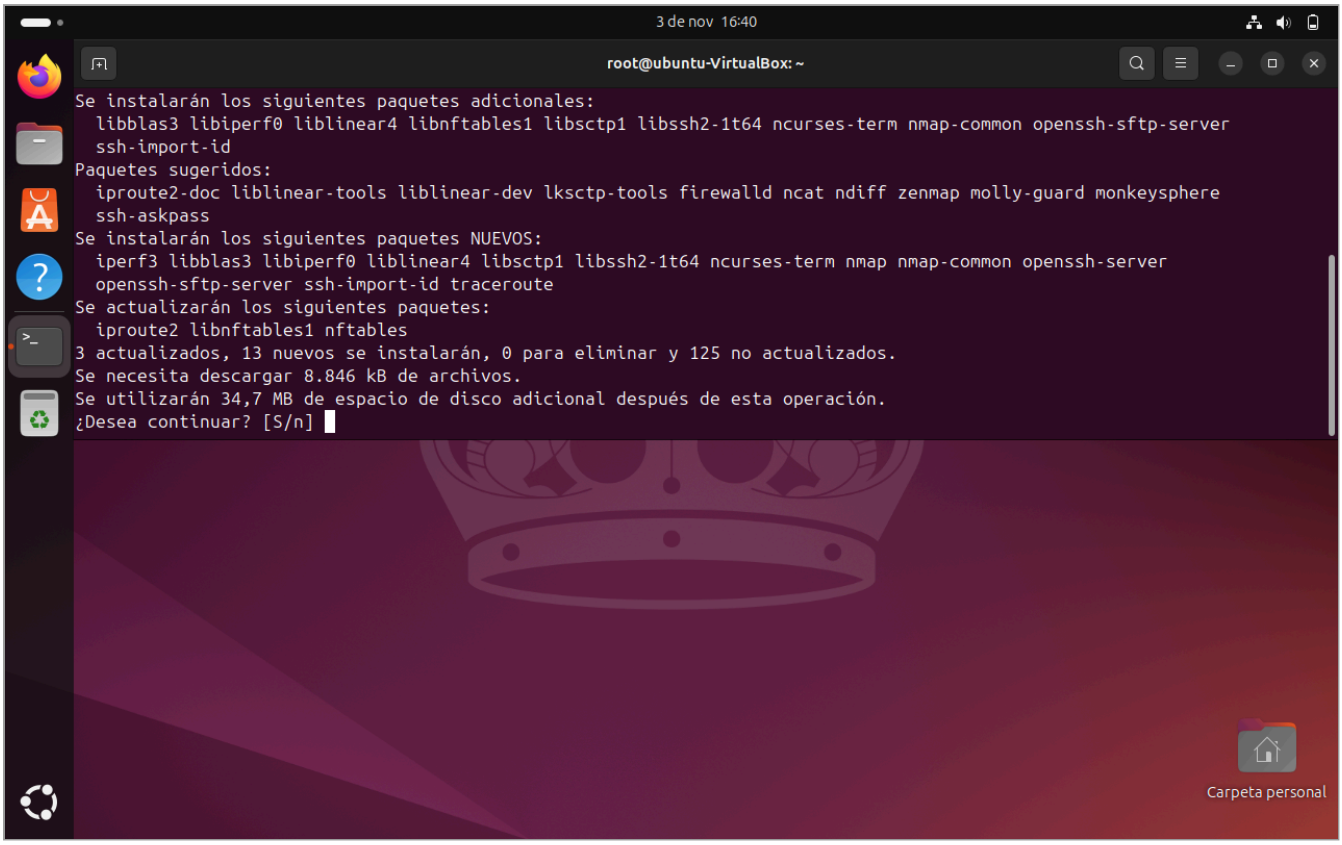
> **Anotar:** 128 paquetes actualizables; sin errores de lock.

`capturas\05-ubuntu-sudo-apt-update.png`

Captura 6 · 3. Preparar la máquina

apt install de las herramientas de la práctica

Se instalan iproute2, nftables, tcpdump, traceroute, nmap, iperf3, time, python3, iputils-ping y openssh-server. apt muestra los paquetes nuevos y los que se actualizan, y pide confirmación [S/n].



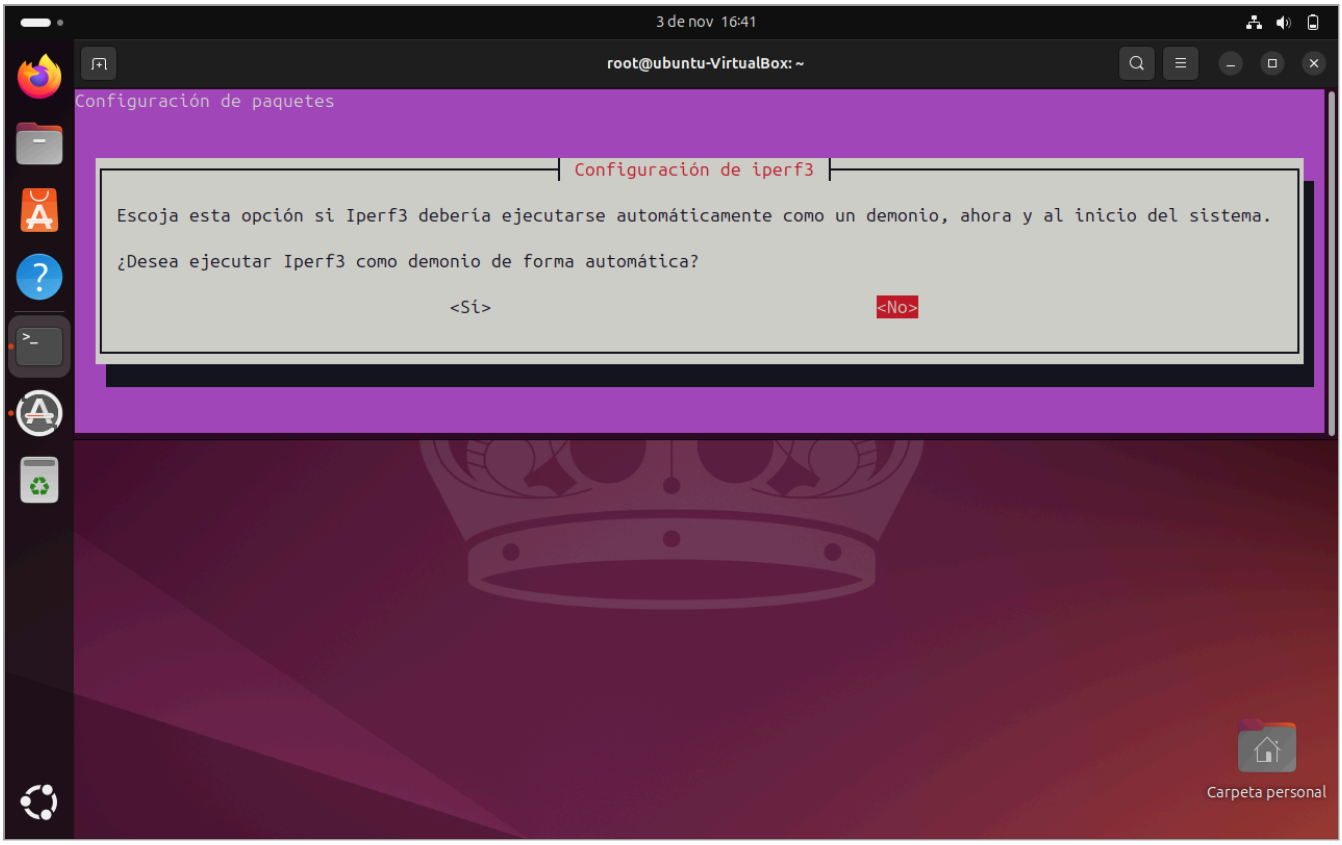
> **Anotar:** 13 nuevos, 3 actualizados (iproute2, libnftables1, nftables).

`capturas\06-ubuntu-apt-install-confirmar.png`

Captura 7 · 3. Preparar la máquina

iperf3 pregunta si iniciarse como demonio: «No»

Durante la instalación, debconf pregunta si iperf3 debe ejecutarse como servicio. Como indica la guía, se responde No.

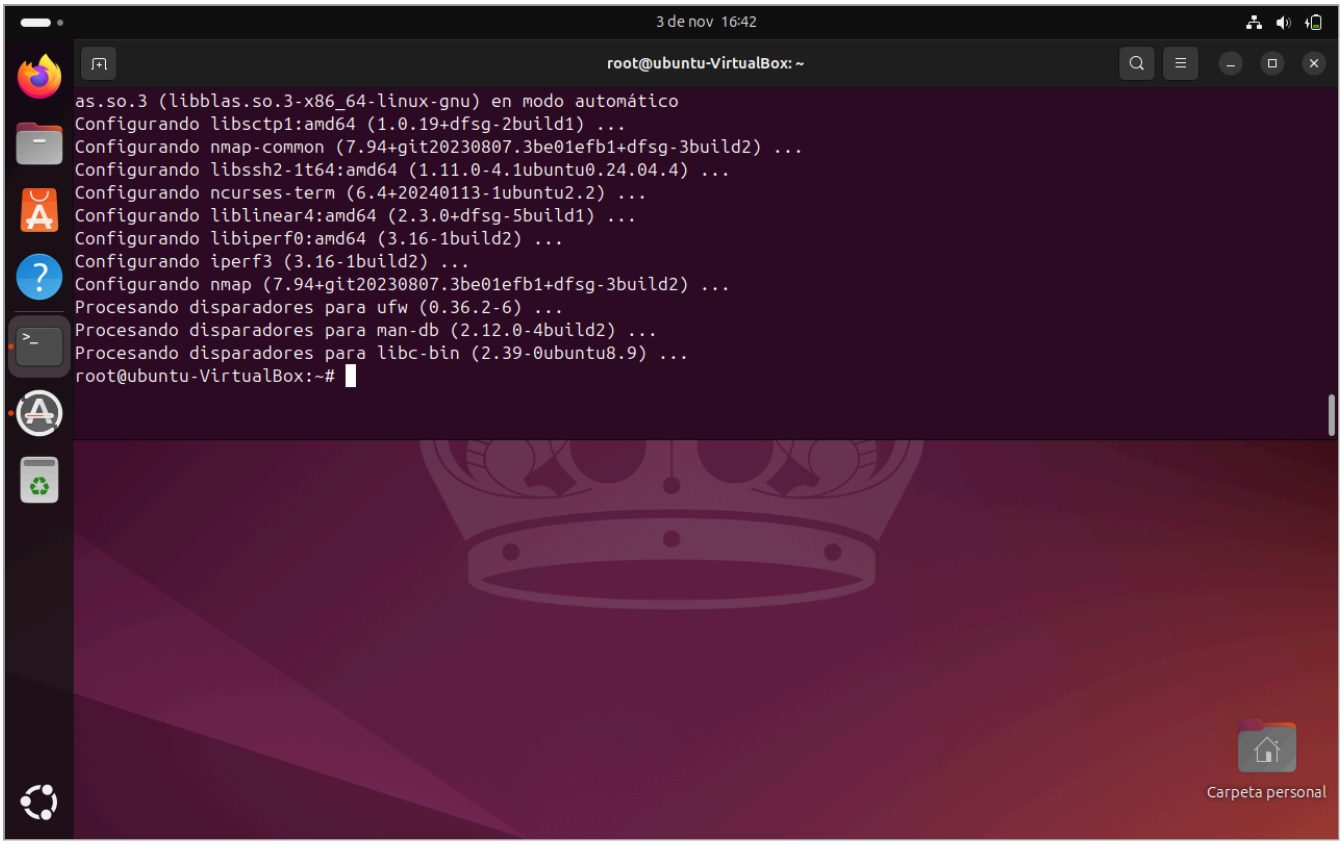


capturas\07-ubuntu-apt-install-iperf3-no.png

Captura 8 · 3. Preparar la máquina

Fin de la instalación

La instalación termina sin errores y vuelve el prompt de root.

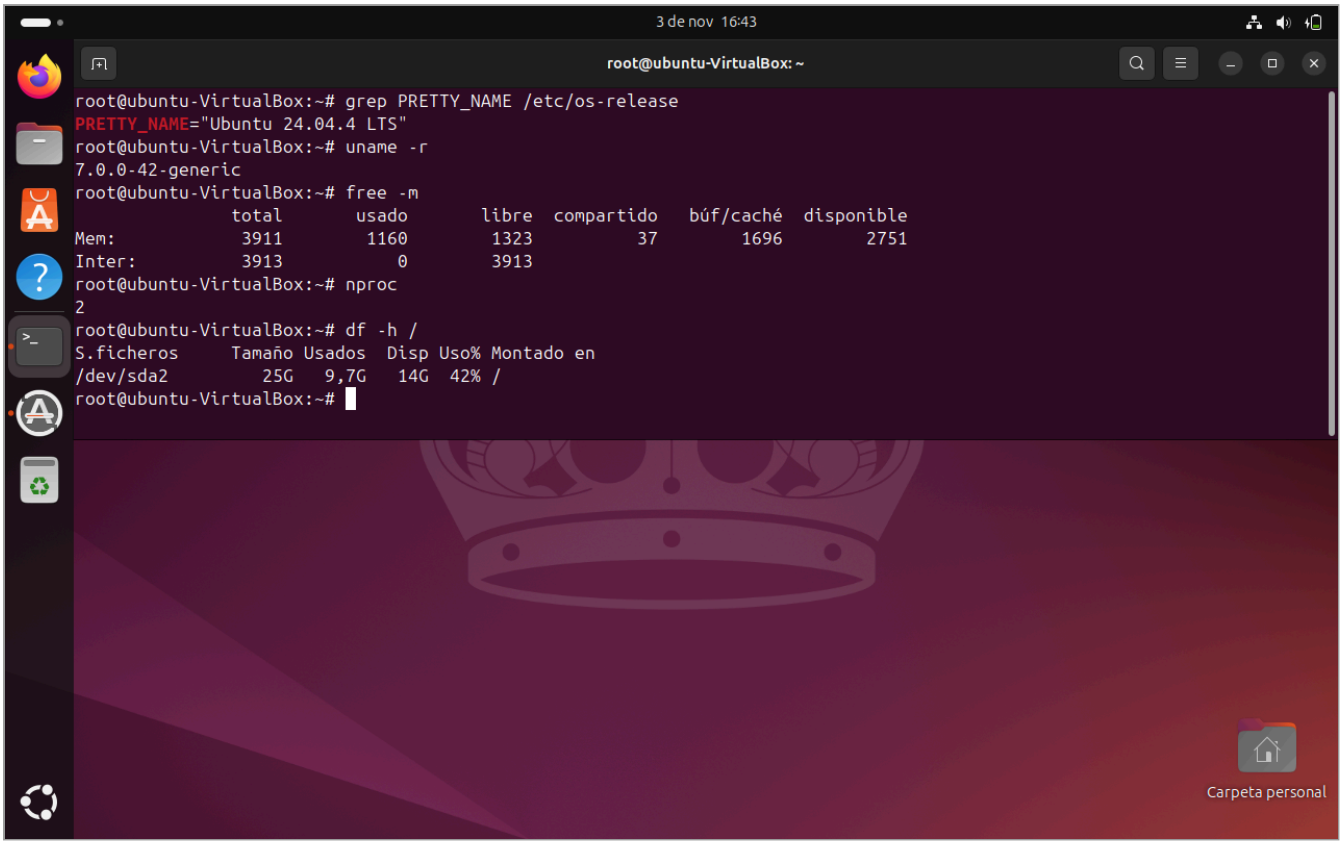


capturas\08-ubuntu-apt-install-fin.png

Captura 9 · 3. Preparar la máquina

Datos de la máquina (Tabla 4)

Versión del sistema, kernel, memoria, procesadores y disco.



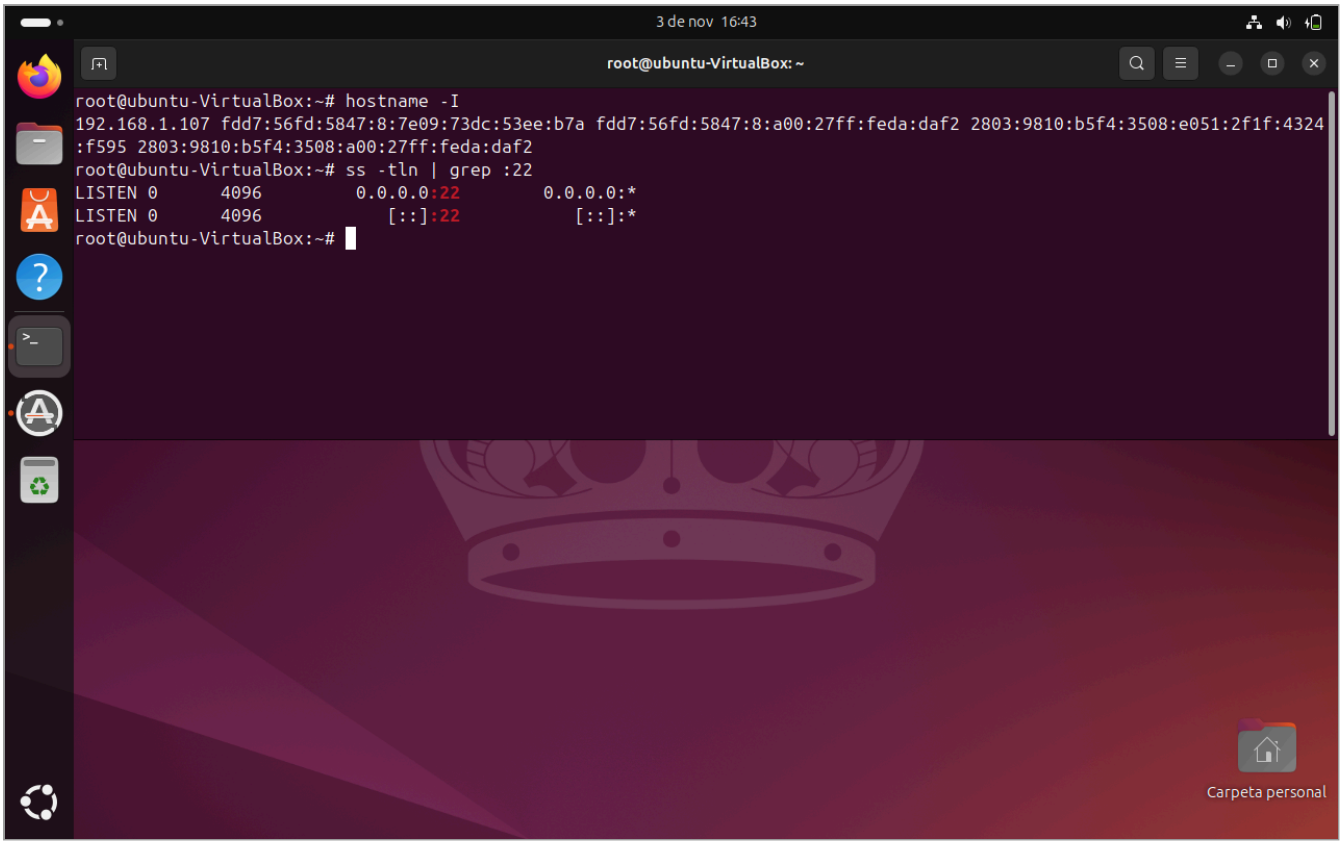
Anotar: Ubuntu 24.04.4 LTS · kernel 7.0.0-42-generic · RAM total 3911 MB (usada 1160 MB) · 2 procesadores · disco /dev/sda2 de 25 GB (9,7 GB usados, 42 %).

`capturas\09-ubuntu-datos-maquina-tabla4.png`

Captura 10 · 4. Conectarse por SSH

IP de la VM y servidor SSH escuchando

hostname -I da la IP que el adaptador puente obtuvo de la red real, y ss -tln confirma que sshd escucha en el puerto 22 (IPv4 e IPv6).



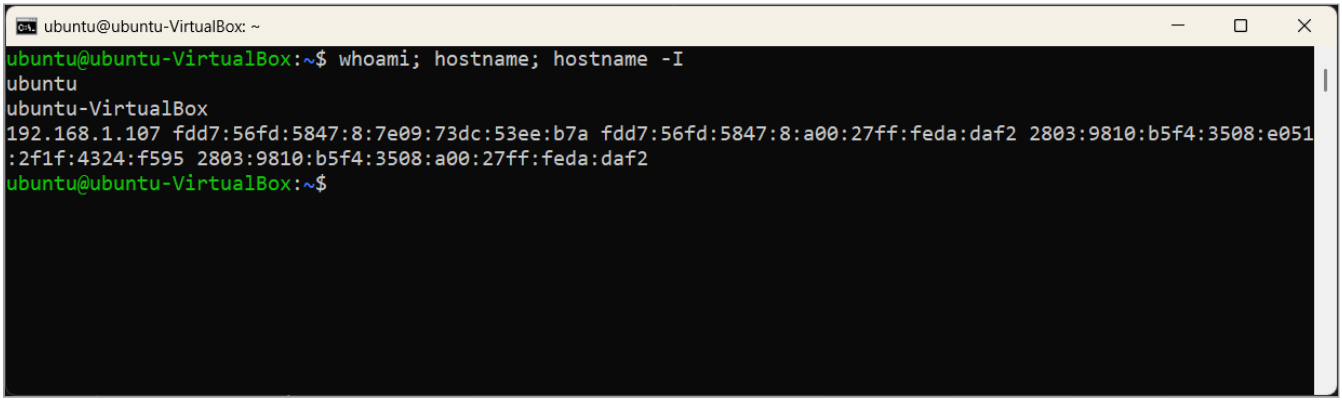
Anotar: IP de la VM: 192.168.1.107. Puerto 22 en LISTEN.

`capturas\10-ubuntu-ip-y-ssh-escuchando.png`

Captura 11 · 4. Conectarse por SSH

Conexión por SSH desde Windows PowerShell

Desde PowerShell en Windows se abre una sesión SSH con la VM (ssh ubuntu@192.168.1.107). Desde acá todos los comandos se ejecutan por SSH.



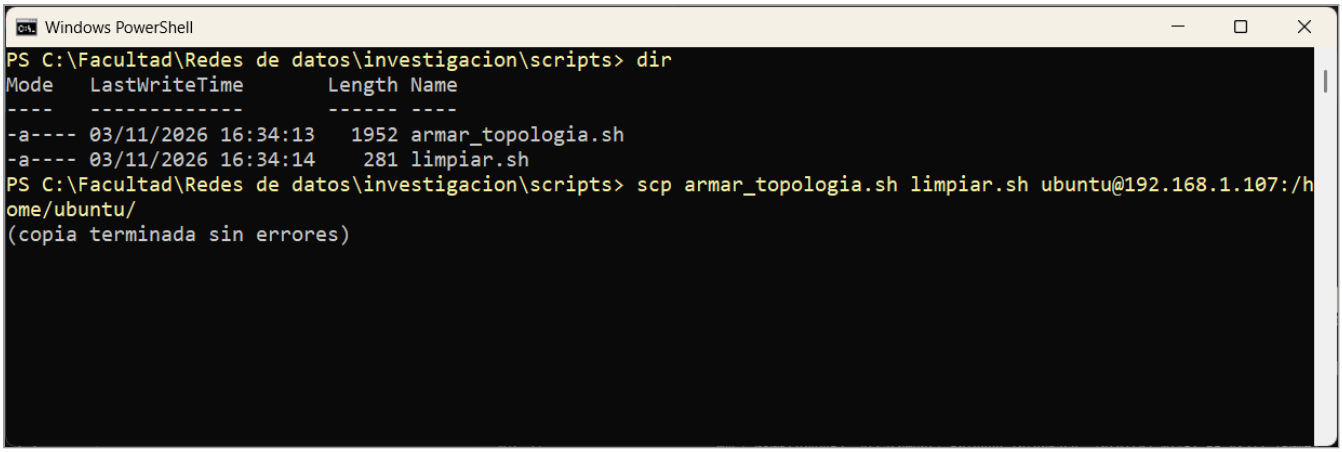
Anotar: Usuario ubuntu en el equipo ubuntu-VirtualBox.

`capturas\11-ubuntu-ssh-desde-windows.png`

Captura 12 · 4. Pasar los scripts

Copia de armar_topologia.sh y limpiar.sh con scp

Desde la carpeta de Windows donde están los scripts del grupo, se copian a /home/ubuntu de la VM con scp.

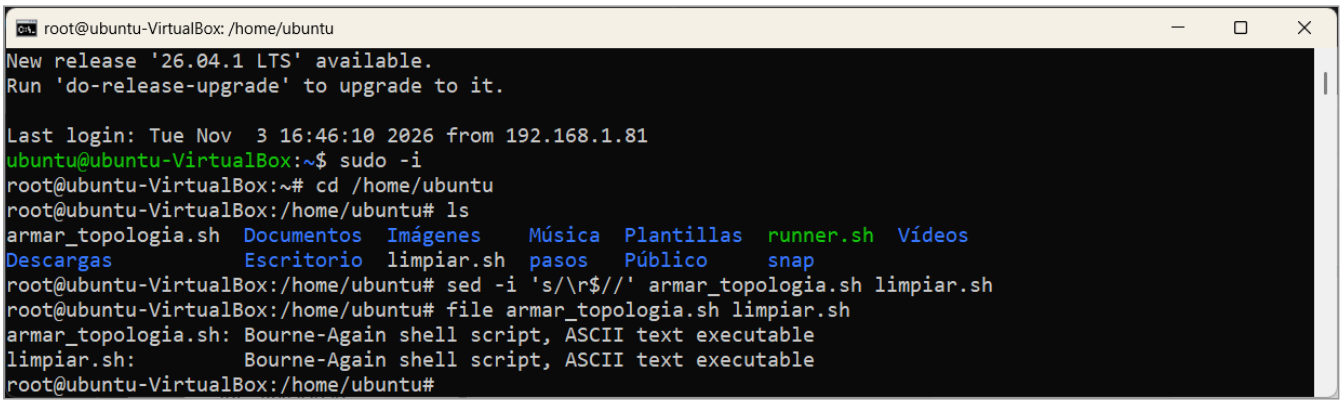


capturas\12-ubuntu-scp-scripts.png

Captura 13 · 4. Pasar los scripts

sudo -i, cd /home/ubuntu y ls

En la sesión SSH se pasa a root, se entra a la carpeta de los scripts y se verifica que estén. Por las dudas se ejecuta el sed que quita los saltos de línea de Windows; file confirma que son scripts de texto ASCII.



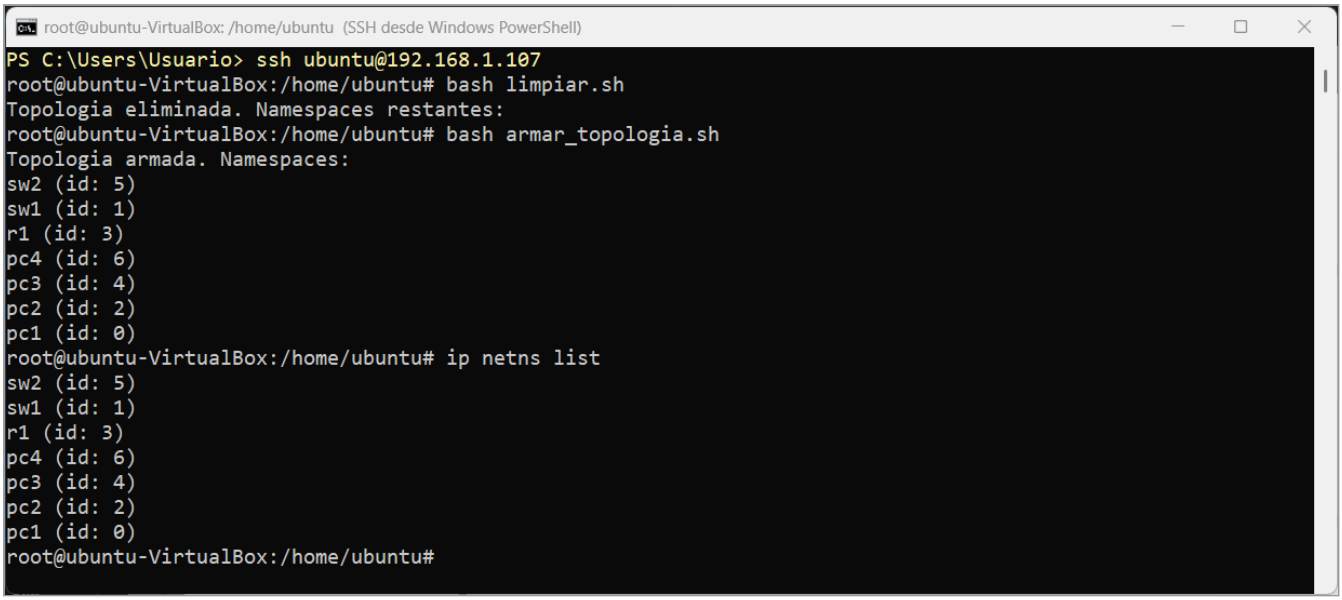
> **Anotar:** Los dos scripts presentes y sin \r.

capturas\13-ubuntu-sudo-cd-ls.png

Captura 14 · Informe 4.1 y 4.2

Armado de la topología

Se ejecuta bash armar_topologia.sh (antes limpiar.sh para empezar de cero) e ip netns list.



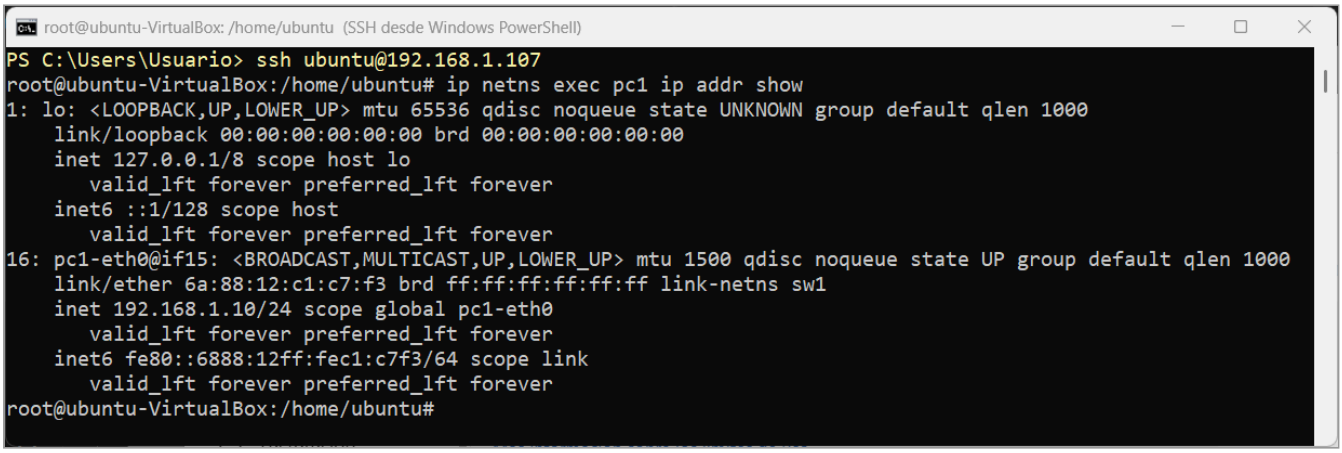
> **Anotar:** 7 namespaces: pc1, pc2, pc3, pc4, r1, sw1 y sw2.

`capturas\14-ubuntu-armar-topologia.png`

Captura 15 · Informe 4.2 — Figura 5 (pc1)

Figura 5 (complemento): ip addr show en pc1

Interfaces del namespace pc1: lo y pc1-eth0 con 192.168.1.10/24.



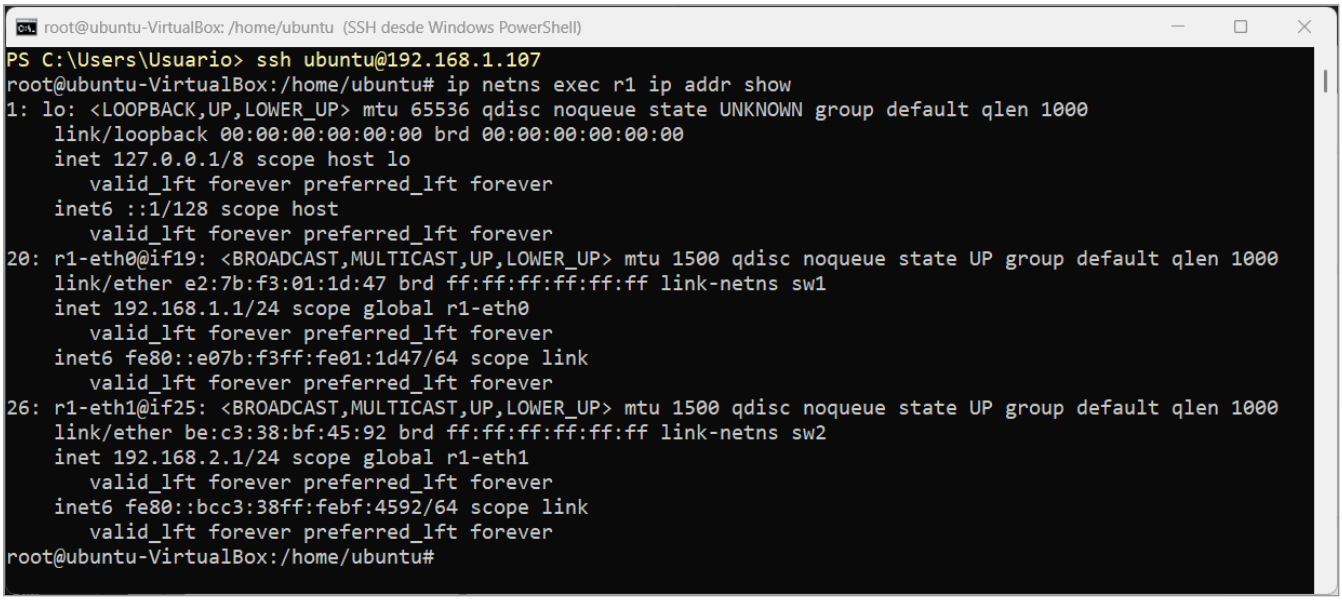
> **Anotar:** Tabla 7 — MAC pc1-eth0: 6a:88:12:c1:c7:f3

`capturas\fig05a-ubuntu-ip-addr-pc1.png`

Captura 16 · Informe 4.2 — Figura 5

Figura 5: ip netns exec r1 ip addr show

El router r1 tiene dos interfaces: r1-eth0 (192.168.1.1/24) conectada a sw1 y r1-eth1 (192.168.2.1/24) conectada a sw2.



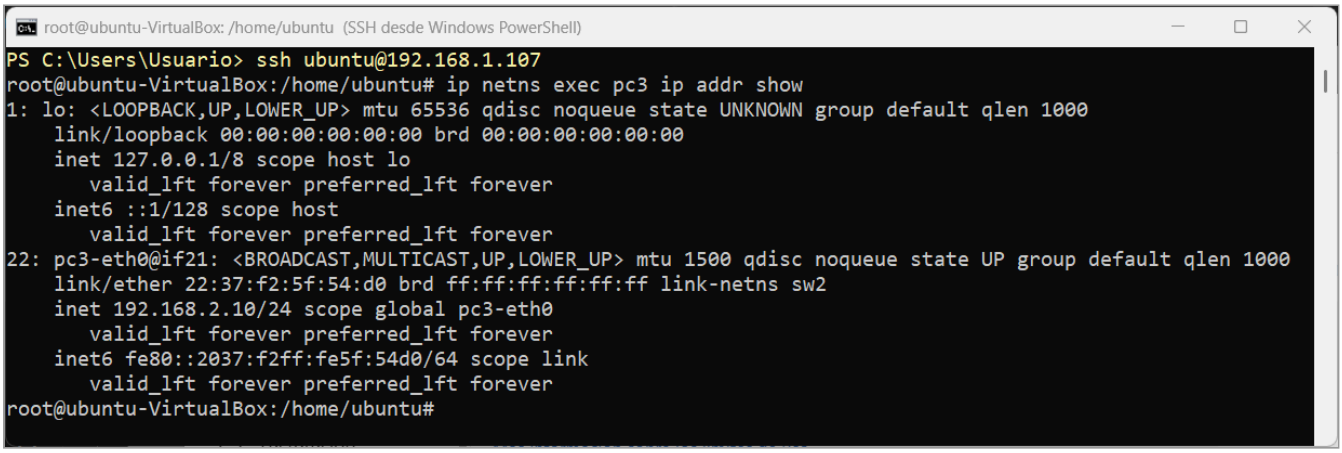
> **Anotar:** Tabla 7 — MAC r1-eth0: e2:7b:f3:01:1d:47 · MAC r1-eth1: be:c3:38:bf:45:92

`capturas\fig05-ubuntu-ip-addr-r1.png`

Captura 17 · Informe 4.2

ip netns exec pc3 ip addr show

Interfaces de pc3: lo y pc3-eth0 con 192.168.2.10/24, conectada a sw2.



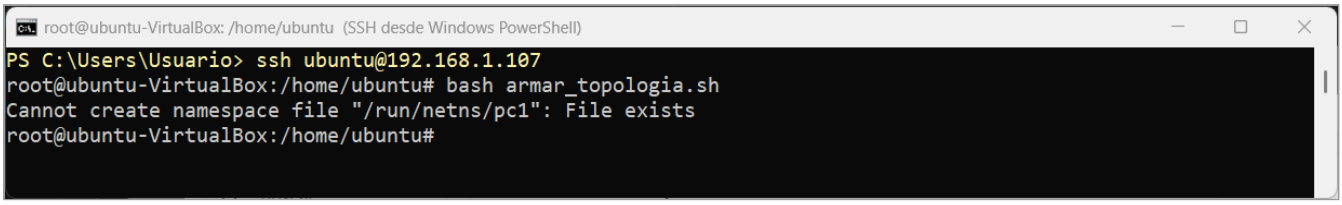
> **Anotar:** Tabla 7 — MAC pc3-eth0: 22:37:f2:5f:54:d0

`capturas\15-ubuntu-ip-addr-pc3.png`

Captura 18 · Informe 4.1 (Si falla)

Armar dos veces: «File exists»

Si se vuelve a armar la topología cuando ya existe, el script se detiene con «File exists», como anticipa la guía. Se resuelve con bash limpiar.sh.



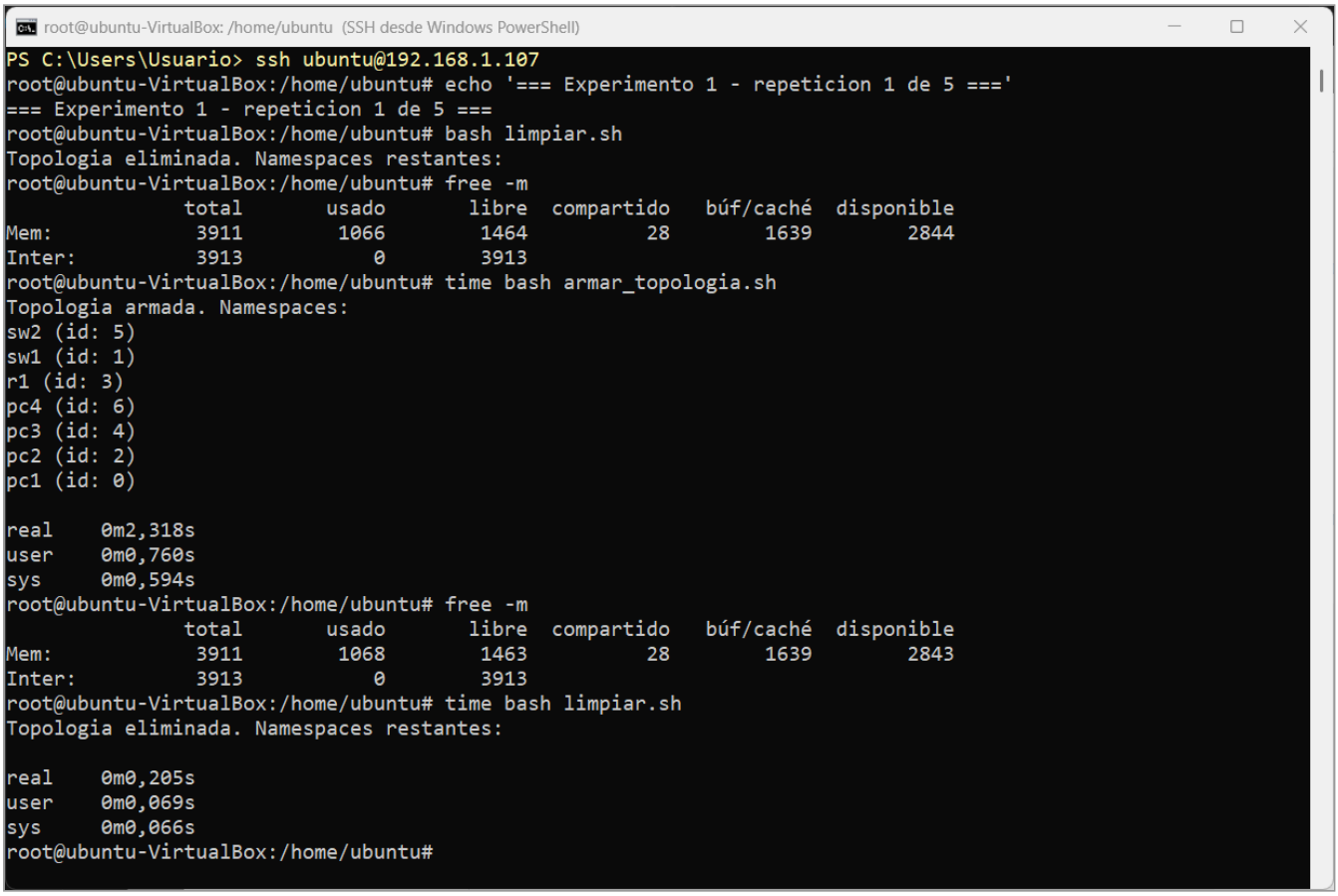
Anotar: Error esperado: Cannot create namespace file "/run/netns/pc1": File exists

capturas\16-ubuntu-armar-file-exists.png

Captura 19 · Informe 4.3 — Experimento 1

Experimento 1, repetición 1 de 5

Ciclo limpiar, free -m, time armar, free -m y time limpiar.



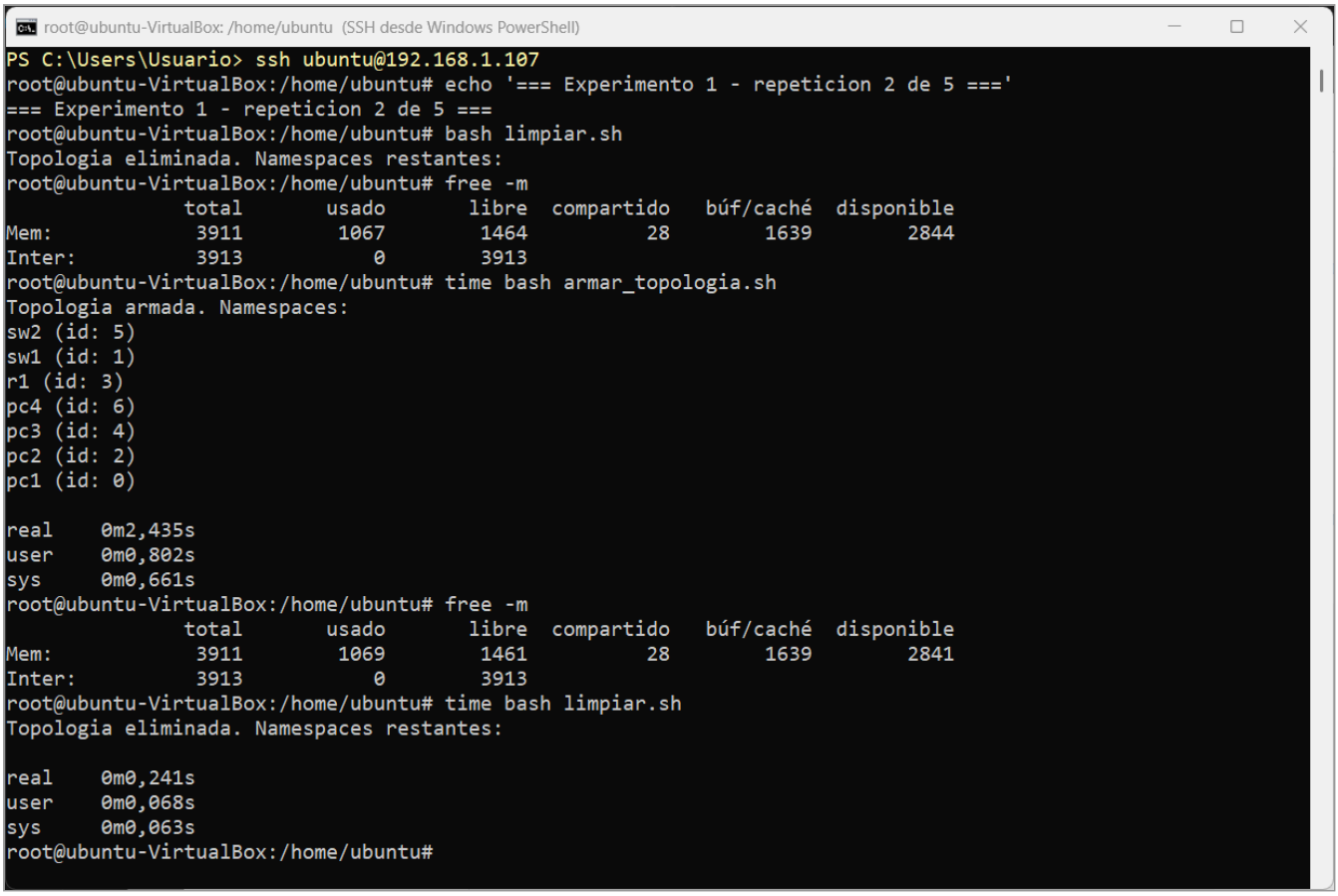
Anotar: Armado 2,318 s · limpieza 0,205 s · used antes 1066 MB, después 1068 MB

capturas\17-ubuntu-exp1-repeticion-1.png

Captura 20 · Informe 4.3 — Experimento 1

Experimento 1, repetición 2 de 5

Mismo ciclo.



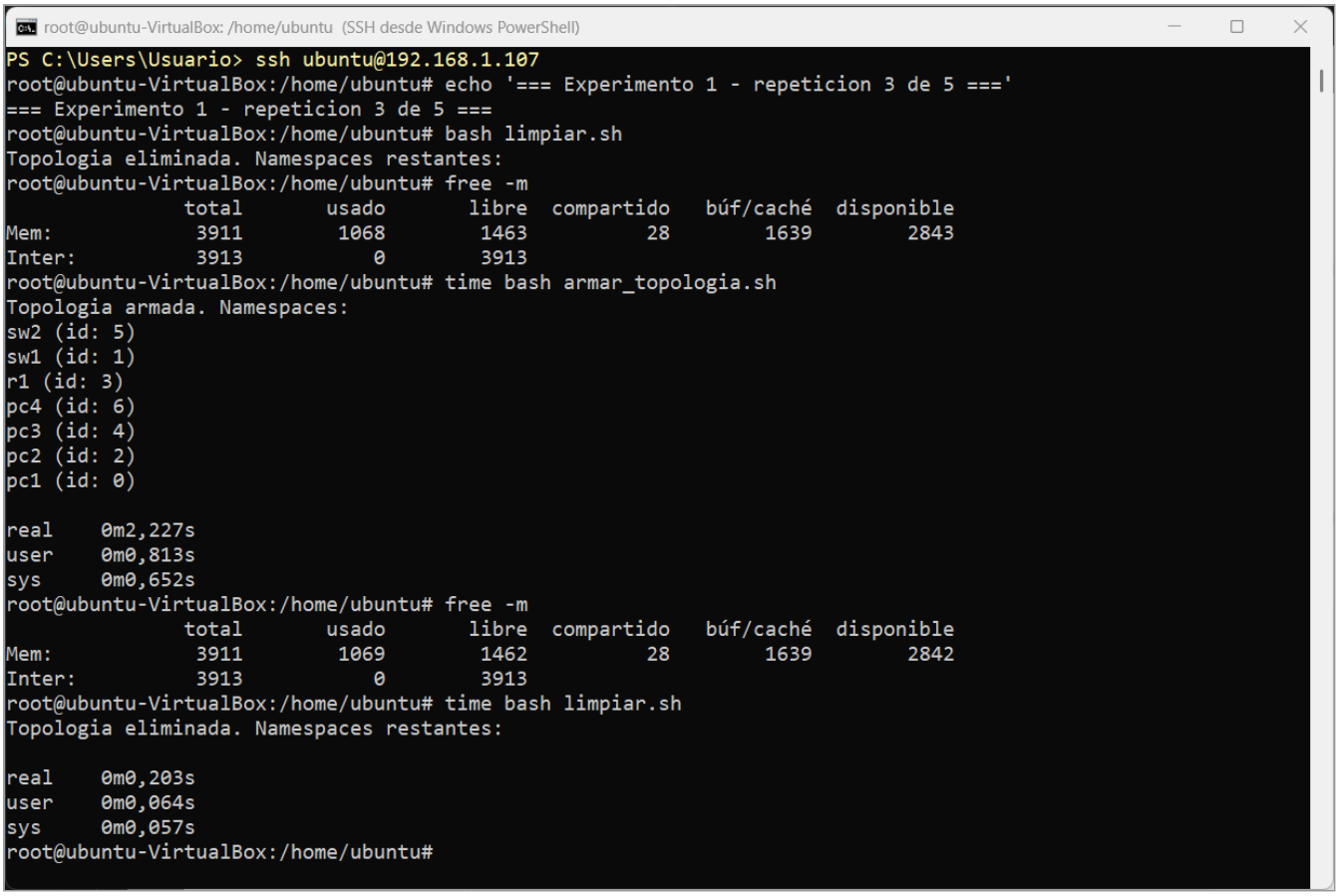
> **Anotar:** Armado 2,435 s · limpieza 0,241 s · used 1067 → 1069 MB

`capturas\17-ubuntu-exp1-repeticion-2.png`

Captura 21 · Informe 4.3 — Experimento 1

Experimento 1, repetición 3 de 5

Mismo ciclo.



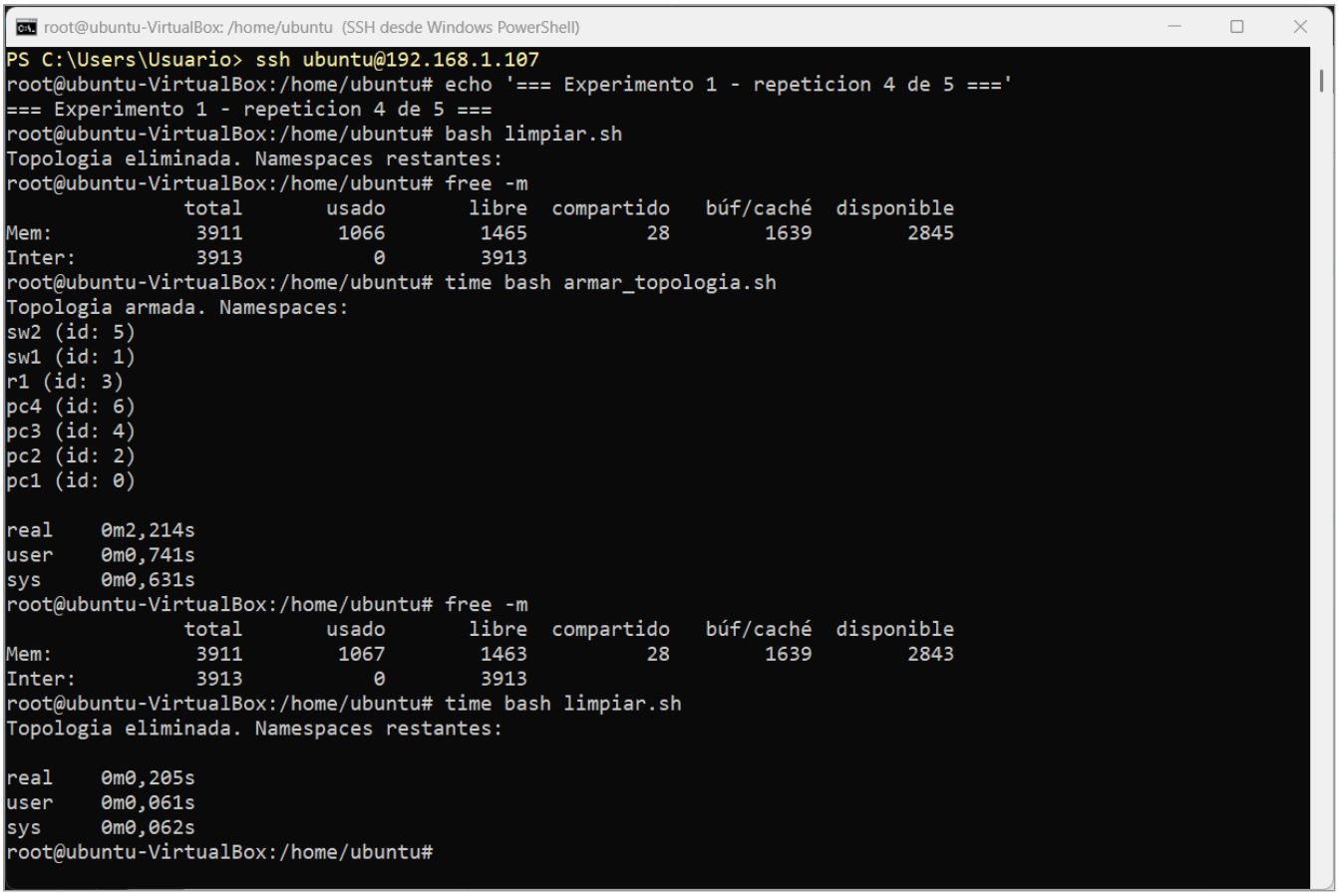
Anotar: Armado 2,227 s · limpieza 0,203 s · used 1068 → 1069 MB

`capturas\17-ubuntu-exp1-repeticion-3.png`

Captura 22 · Informe 4.3 — Experimento 1

Experimento 1, repetición 4 de 5

Mismo ciclo.



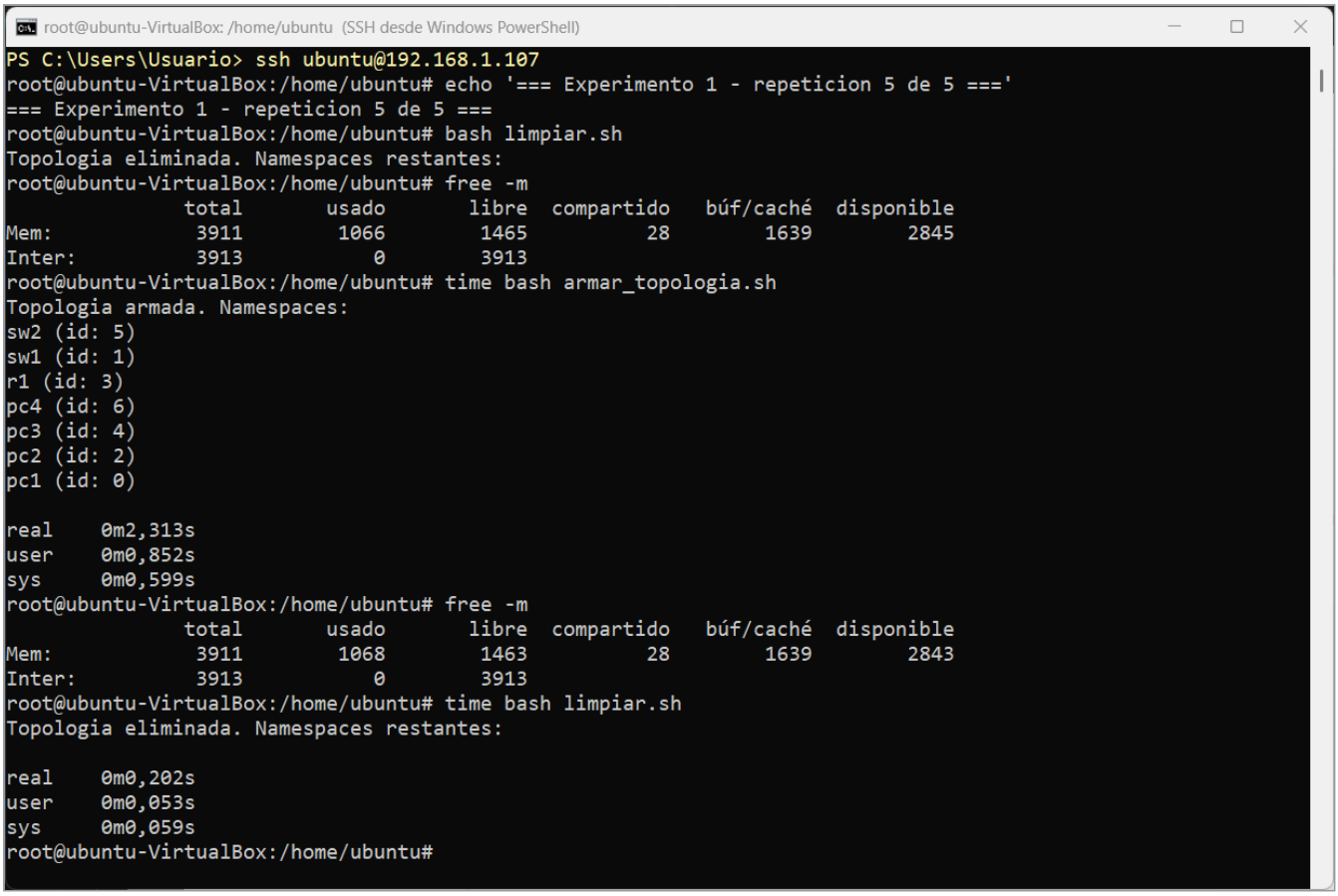
> **Anotar:** Armado 2,214 s · limpieza 0,205 s · used 1066 → 1067 MB

`capturas\17-ubuntu-exp1-repeticion-4.png`

Captura 23 · Informe 4.3 — Experimento 1

Experimento 1, repetición 5 de 5

Mismo ciclo.



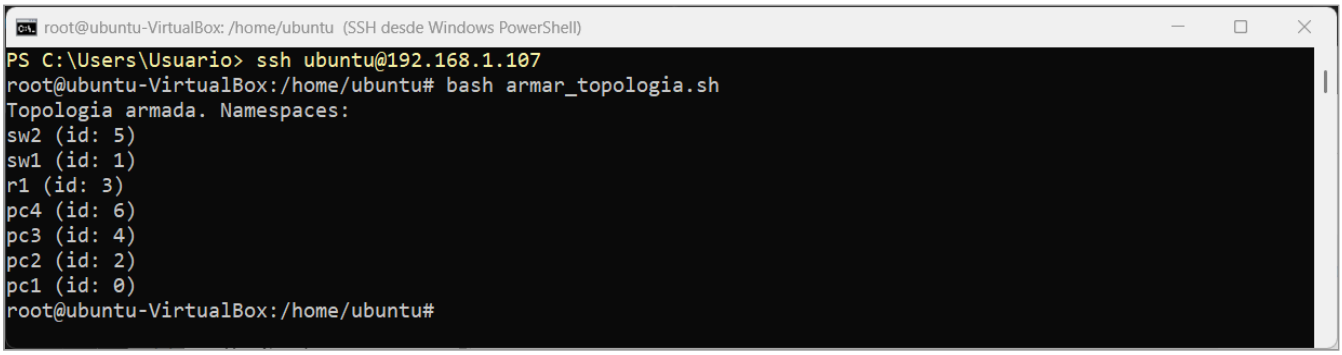
> **Anotar:** Armado 2,313 s · limpieza 0,202 s · used 1066 → 1068 MB

capturas\17-ubuntu-exp1-repeticion-5.png

Captura 24 · Informe 4.3 — Experimento 1

Topología armada otra vez para seguir

Después de la quinta repetición se vuelve a armar la topología para los experimentos siguientes.

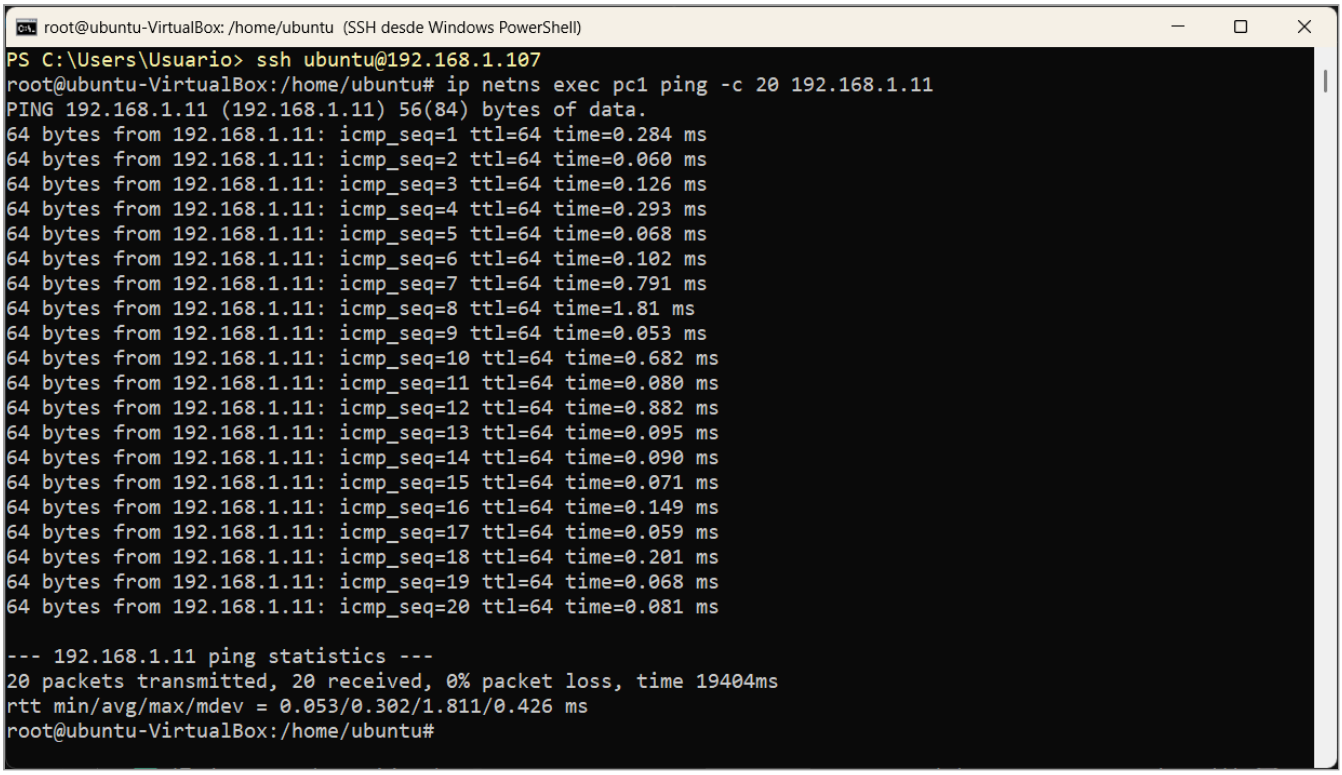


capturas\18-ubuntu-exp1-rearmar.png

Captura 25 · Informe 4.4 a) — Experimento 2

Ping dentro de la misma red (pc1 → pc2)

pc1 hace ping a pc2 (192.168.1.11), que está en la misma red A: llega directo a través del switch sw1.



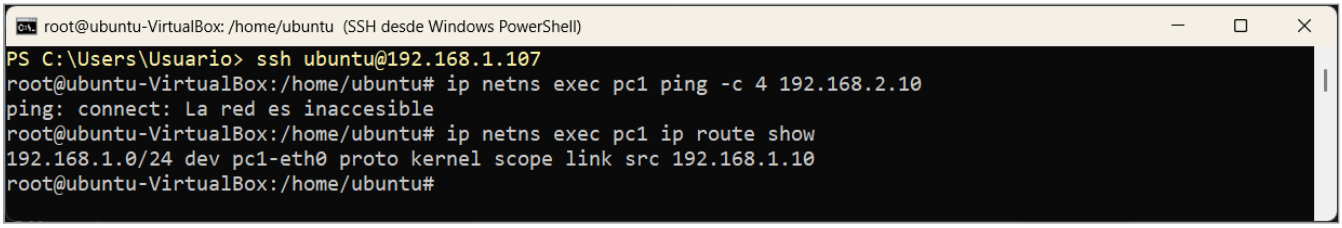
> **Anotar:** 20 enviados, 20 recibidos, 0 % de pérdida · rtt min/avg/max = 0,053/0,302/1,811 ms

`capturas\19-ubuntu-exp2a-ping-misma-red.png`

Captura 26 · Informe 4.4 a) — Figura 6

Figura 6: ping a otra red sin rutas e ip route show

pc1 no tiene ruta hacia 192.168.2.0/24: su tabla de rutas solo tiene la red directamente conectada, y el ping falla de inmediato.



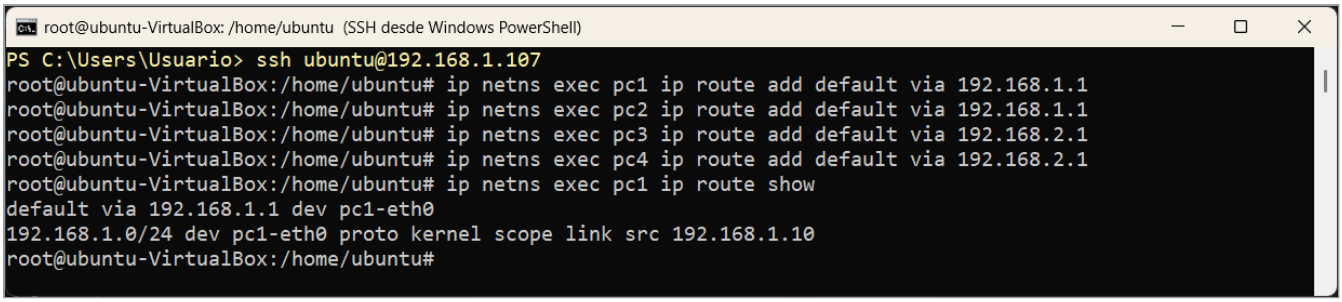
> **Anotar:** Mensaje de error: «ping: connect: La red es inaccesible» (Network is unreachable)

`capturas\fig06-ubuntu-ping-sin-ruta.png`

Captura 27 · Informe 4.4 b)

Carga de las rutas por defecto

Se agrega en cada PC una ruta por defecto hacia su gateway: r1 (192.168.1.1 para la red A y 192.168.2.1 para la red B).



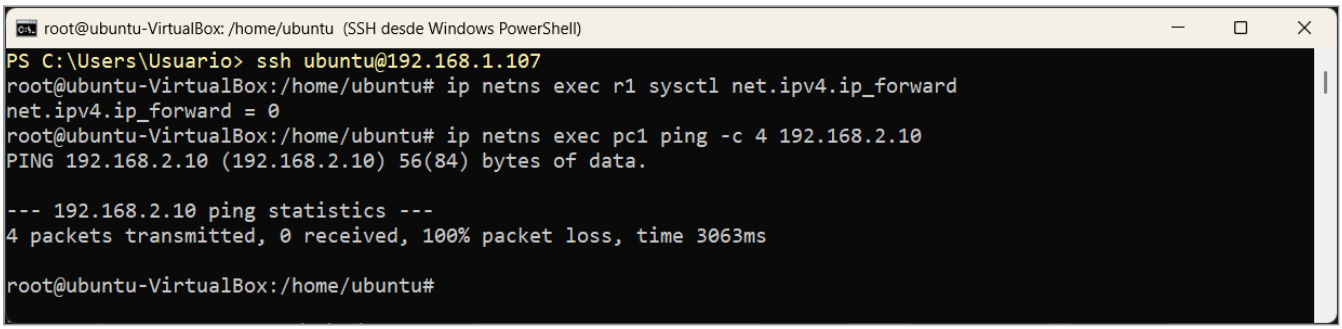
> **Anotar:** pc1: default via 192.168.1.1 dev pc1-eth0

`capturas\20-ubuntu-exp2b-rutas-por-defecto.png`

Captura 28 · Informe 4.4 b) — Figura 7

Figura 7: ip_forward = 0 y ping con pérdida

Con las rutas cargadas pero el reenvío del router desactivado, r1 recibe los paquetes y no los reenvía.



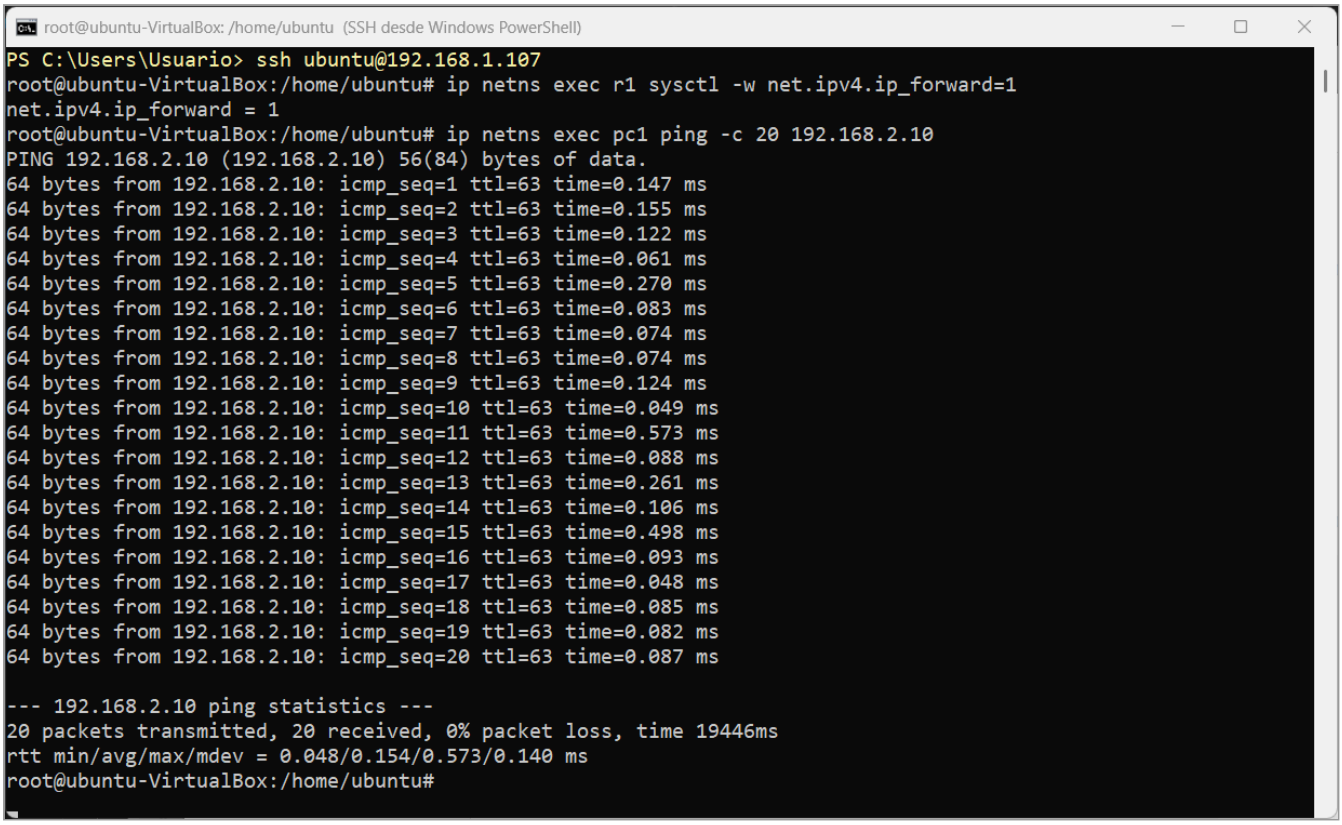
> **Anotar:** ip_forward = 0 · 4 enviados, 0 recibidos, 100 % de pérdida

`capturas\fig07-ubuntu-ip-forward-0-ping-perdida.png`

Captura 29 · Informe 4.4 c) — Figura 8

Figura 8: reenvío activado y ping exitoso (repetición 1)

Se activa net.ipv4.ip_forward=1 en r1, y pc1 llega a pc3 en la otra red. El TTL baja de 64 a 63 porque se pasa por un router.



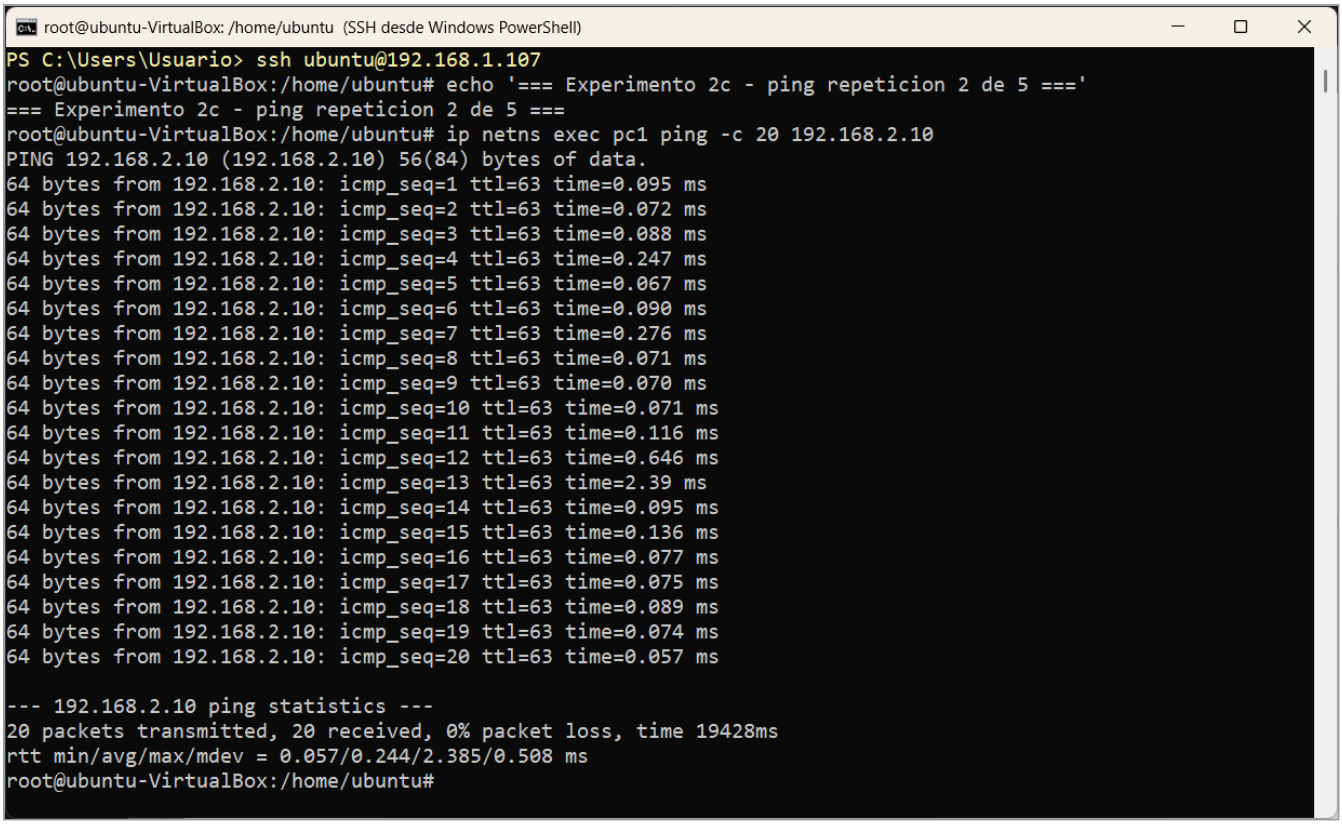
> **Anotar:** 0 % de pérdida · rtt 0,048/0,154/0,573 ms · ttl = 63

`capturas\fig08-ubuntu-ip-forward-1-ping-exitoso.png`

Captura 30 · Informe 4.4 c)

Ping entre redes, repetición 2 de 5

Repetición para promediar el rtt avg.



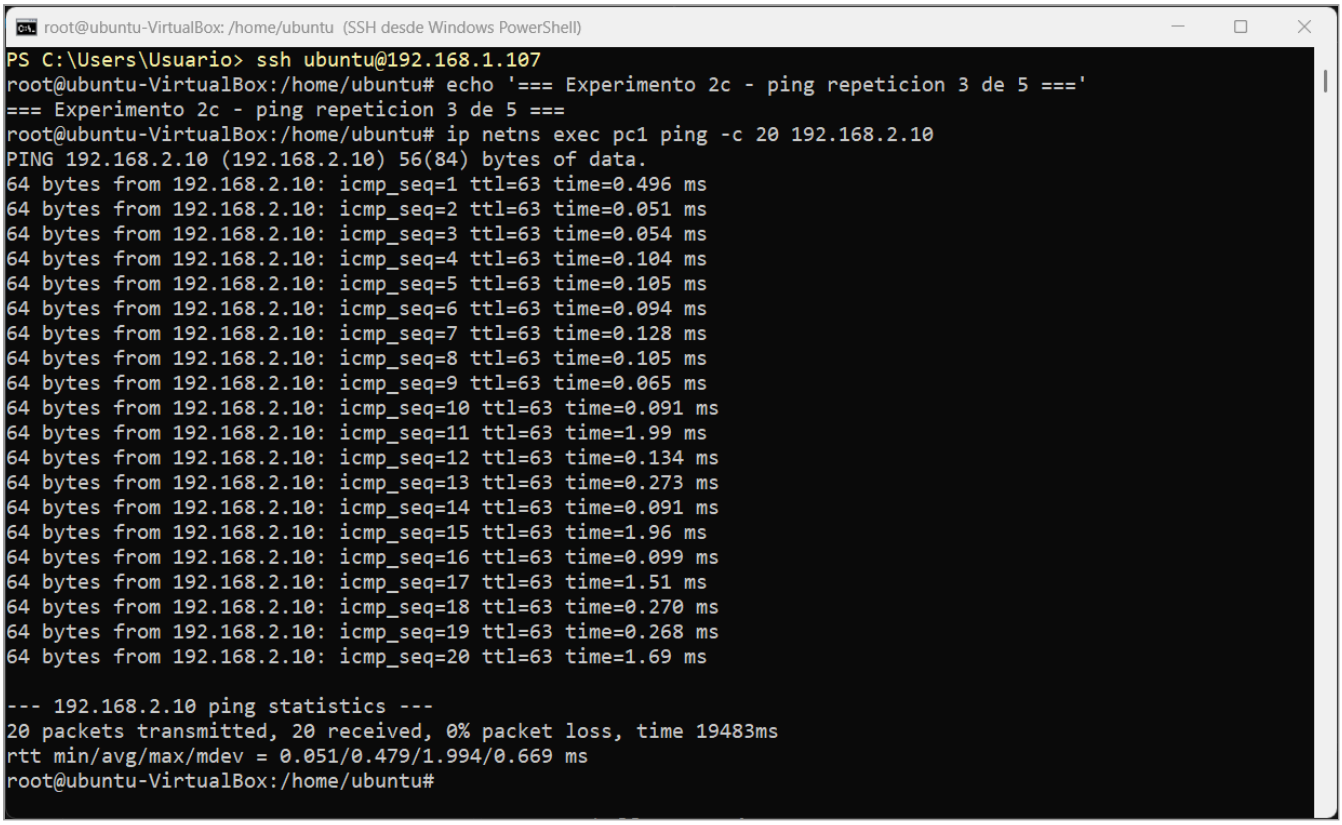
> **Anotar:** 0 % · rtt 0,057/0,244/2,385 ms

`capturas\21-ubuntu-exp2c-ping-repeticion-2.png`

Captura 31 · Informe 4.4 c)

Ping entre redes, repetición 3 de 5

Repetición para promediar el rtt avg.



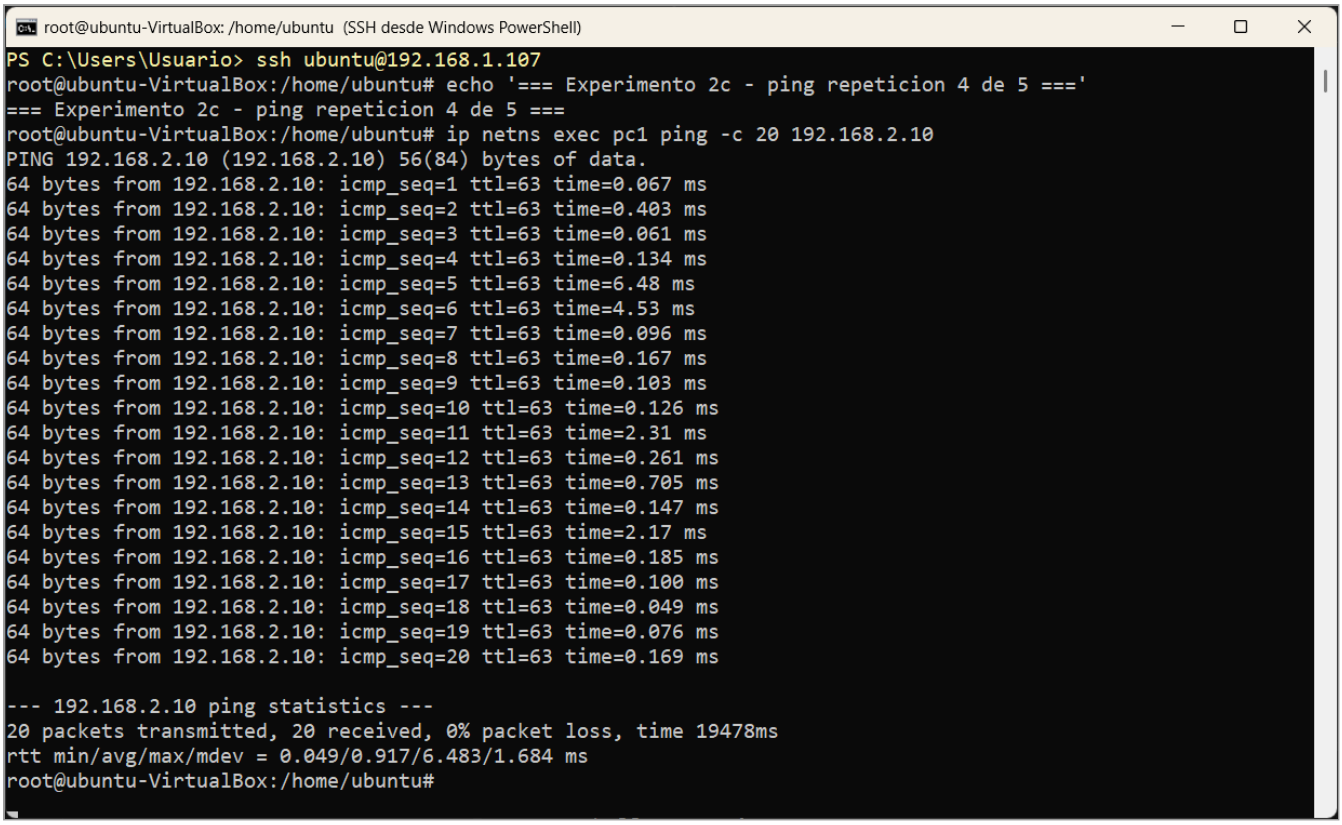
> **Anotar:** 0 % · rtt 0,051/0,479/1,994 ms

`capturas\21-ubuntu-exp2c-ping-repeticion-3.png`

Captura 32 · Informe 4.4 c)

Ping entre redes, repetición 4 de 5

Repetición para promediar el rtt avg.



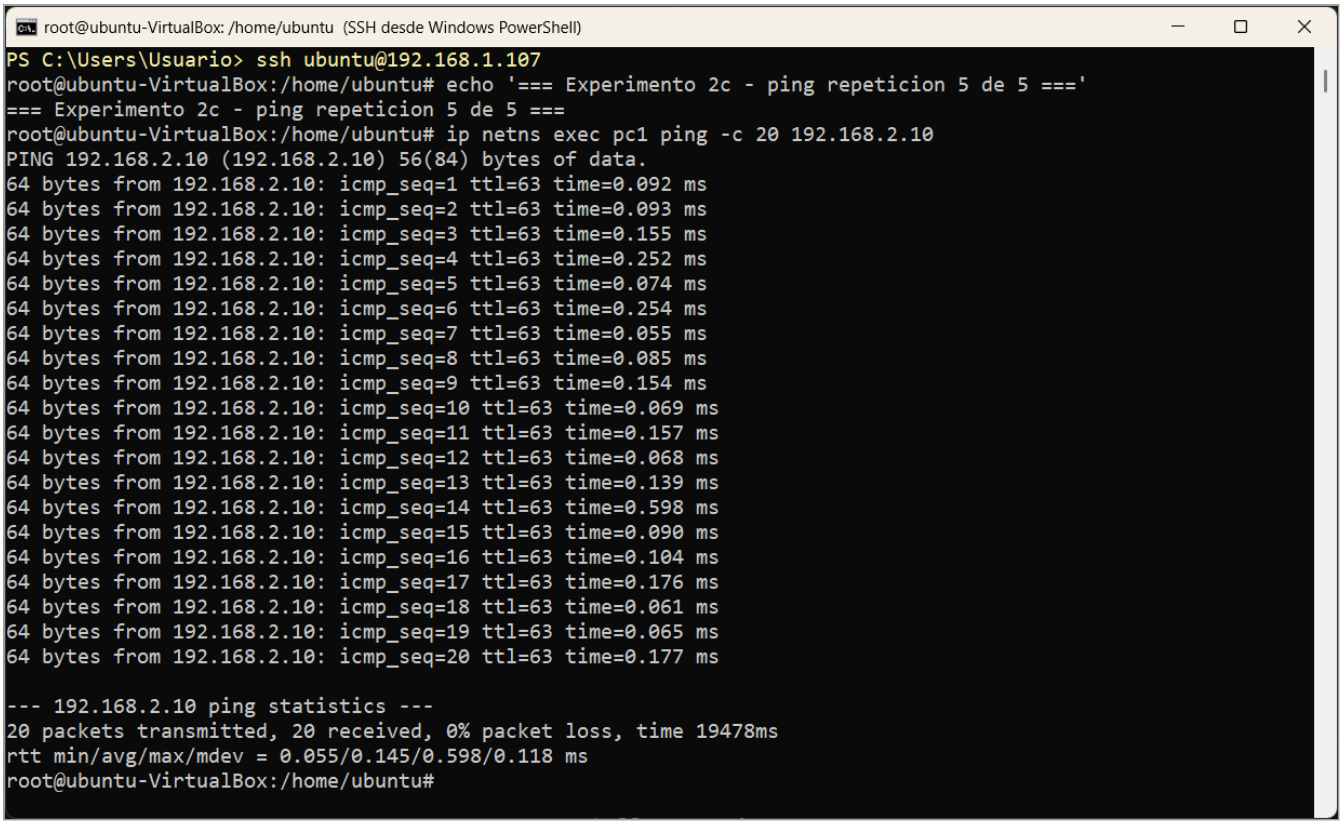
> **Anotar:** 0 % · rtt 0,049/0,917/6,483 ms

`capturas\21-ubuntu-exp2c-ping-repeticion-4.png`

Captura 33 · Informe 4.4 c)

Ping entre redes, repetición 5 de 5

Repetición para promediar el rtt avg.



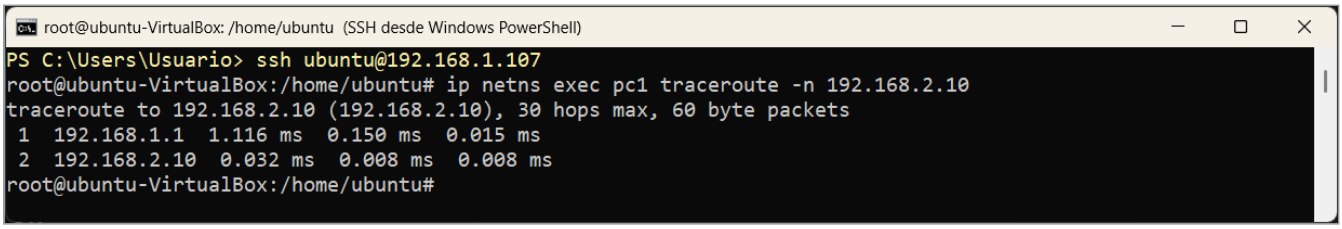
> **Anotar:** 0 % · rtt 0,055/0,145/0,598 ms

`capturas\21-ubuntu-exp2c-ping-repeticion-5.png`

Captura 34 · Informe 4.4 c) — Figura 9

Figura 9: traceroute de pc1 a pc3

El traceroute muestra el camino: primer salto r1 (192.168.1.1) y segundo salto el destino pc3.



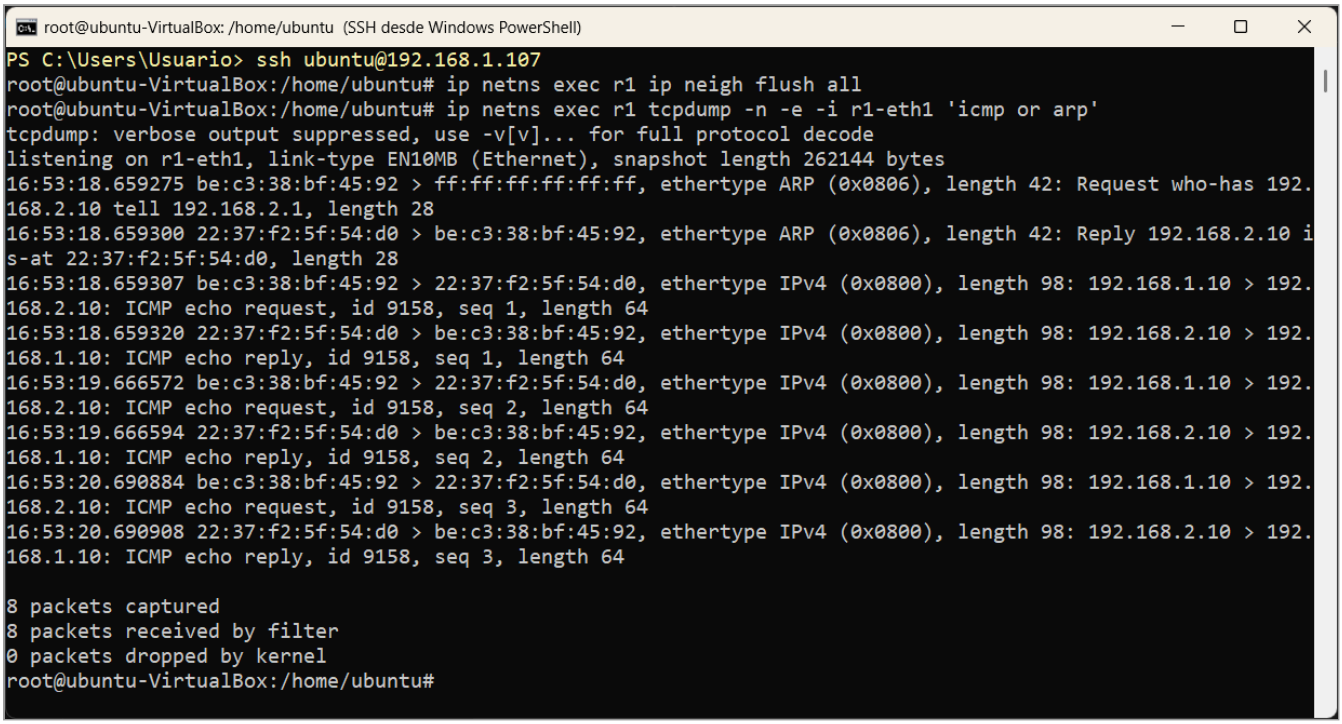
> **Anotar:** 2 saltos: 192.168.1.1 → 192.168.2.10

`capturas\fig09-ubuntu-traceroute.png`

Captura 35 · Informe 4.5 — Figura 10

Figura 10: tcpdump en r1-eth1 (terminal 1)

Después de vaciar la tabla ARP de r1, se captura en r1-eth1 mientras pc1 hace 3 pings a pc3. Primero r1 pregunta por ARP quién tiene 192.168.2.10 y después pasan los echo request/reply. Captura detenida con Ctrl+C (SIGINT).



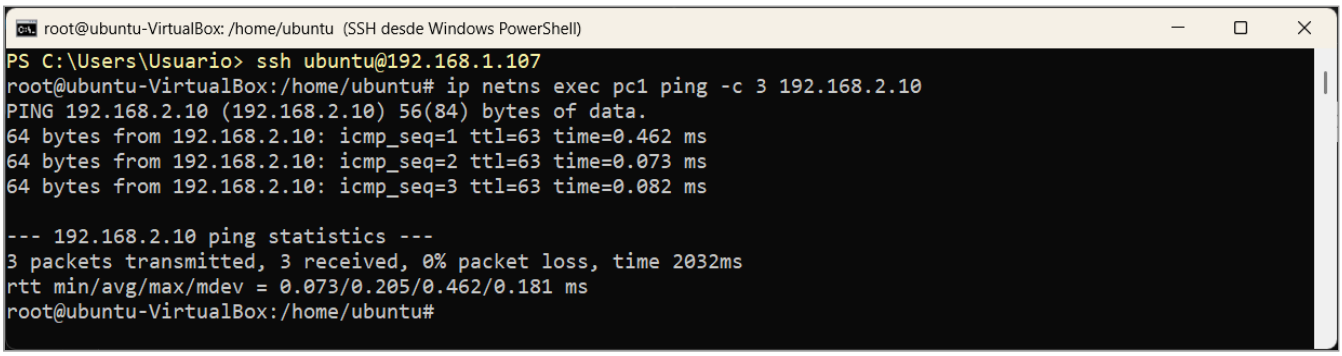
> **Anotar:** Tabla 10: 2 ARP + 6 ICMP = 8 paquetes · MAC origen r1-eth1 be:c3:38:bf:45:92 → destino pc3-eth0 22:37:f2:5f:54:d0 (los paquetes llevan IP de pc1, pero MAC de r1)

`capturas\fig10-ubuntu-tcpdump-r1-eth1.png`

Captura 36 · Informe 4.5

Terminal 2: ping -c 3 mientras corre tcpdump

En la segunda terminal pc1 envía 3 pings a pc3 mientras la primera captura.



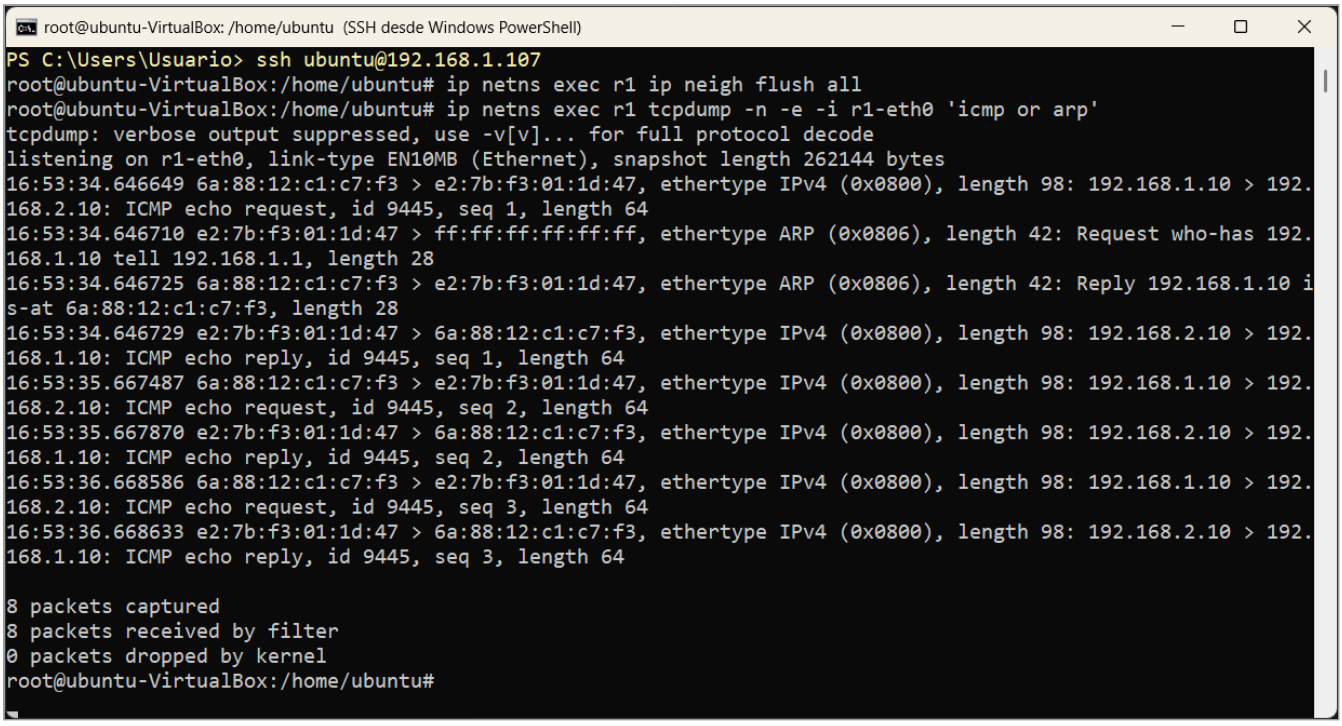
> **Anotar:** 3 enviados, 3 recibidos, 0 % de pérdida

`capturas\22-ubuntu-tcpdump-terminal2-ping.png`

Captura 37 · Informe 4.5 (opcional)

tcpdump en r1-eth0 para comparar las MAC

La misma prueba del lado de la red A. Ahora la MAC de origen del request es la de pc1 y la de destino la de r1-eth0: la MAC cambia en cada salto y las IP no.



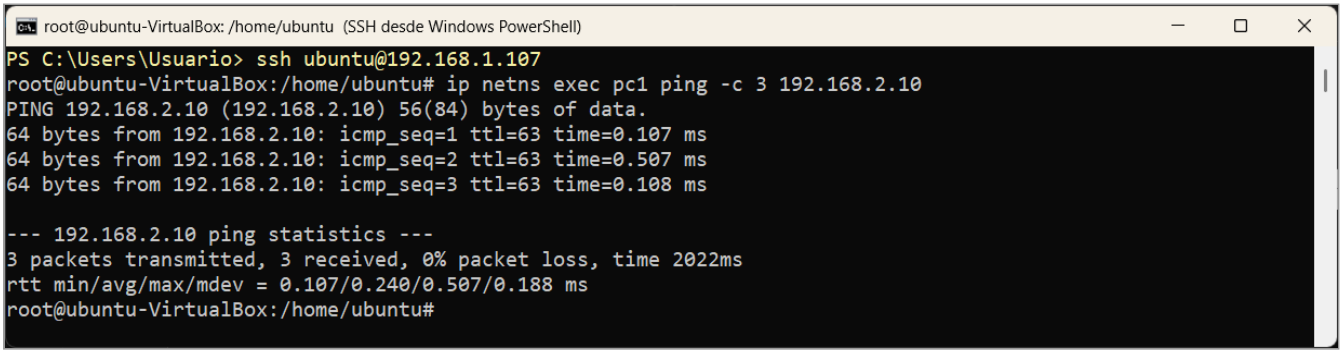
> **Anotar:** 8 paquetes (2 ARP + 6 ICMP) · pc1-eth0 6a:88:12:c1:c7:f3 → r1-eth0 e2:7b:f3:01:1d:47

`capturas\23-ubuntu-tcpdump-r1-eth0-opcional.png`

Captura 38 · Informe 4.5 (opcional)

Terminal 2 de la prueba opcional

Ping de pc1 a pc3 durante la captura en r1-eth0.



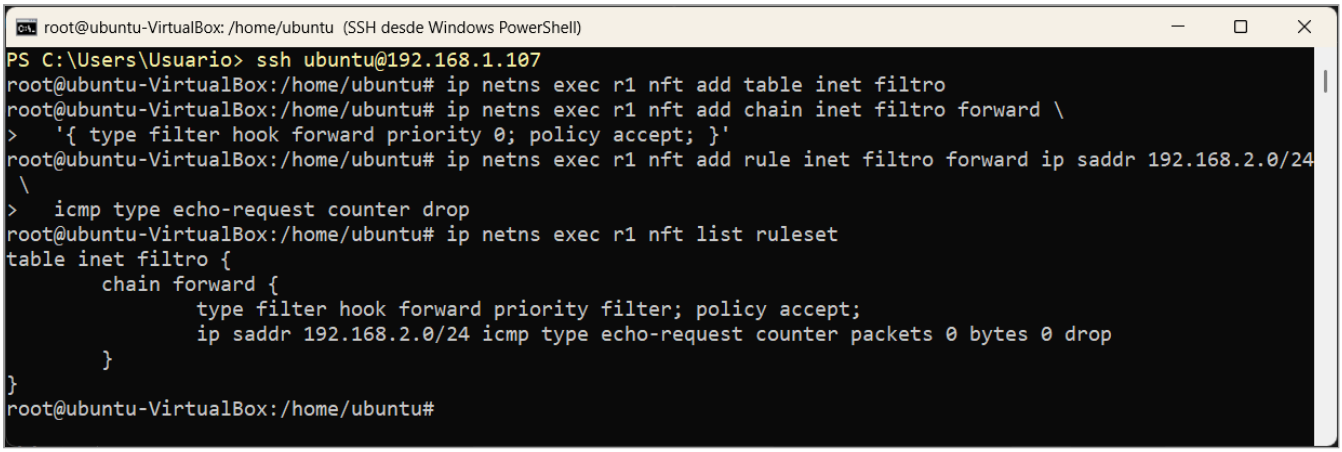
> **Anotar:** 0 % de pérdida

`capturas\24-ubuntu-tcpdump-r1-eth0-terminal2-ping.png`

Captura 39 · Informe 4.6 a) — Figura 11

Figura 11: nft list ruleset con la regla anti-ping

Se crea la tabla inet filtro con una cadena en el hook forward y una regla que descarta los echo-request que vienen de la red 192.168.2.0/24, con contador.



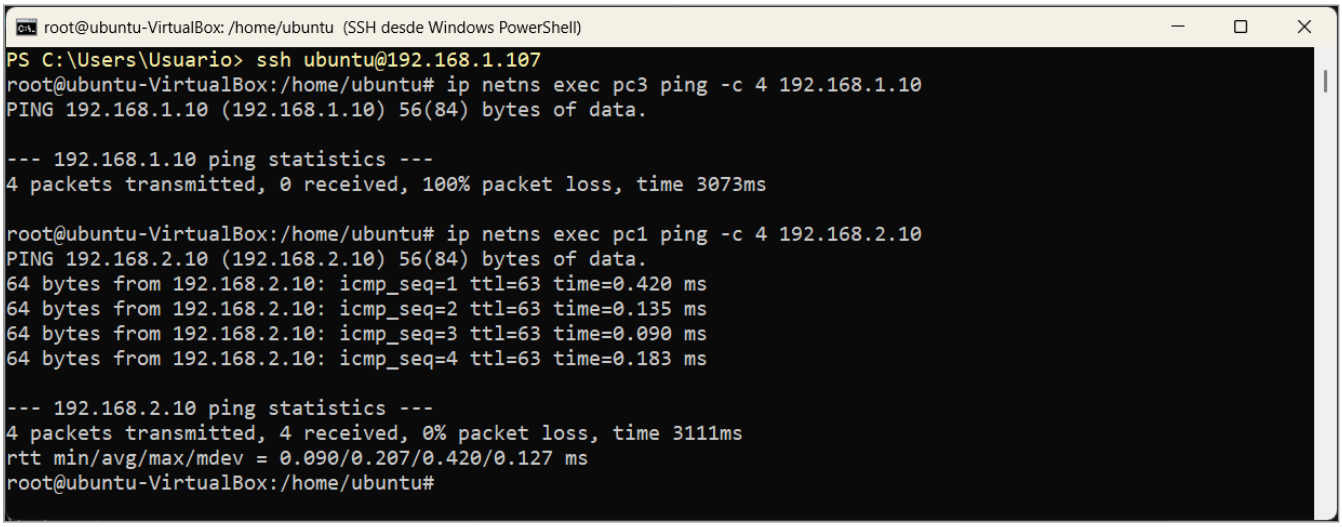
> **Anotar:** counter packets 0 bytes 0 (todavía sin tráfico)

`capturas\fig11-ubuntu-nft-ruleset-icmp.png`

Captura 40 · Informe 4.6 a) — Figura 12

Figura 12: ping de pc3 a pc1 y de pc1 a pc3 con firewall

El ping que inicia la red B (pc3 → pc1) se bloquea. El que inicia la red A (pc1 → pc3) funciona, porque el echo-reply de pc3 no coincide con la regla.



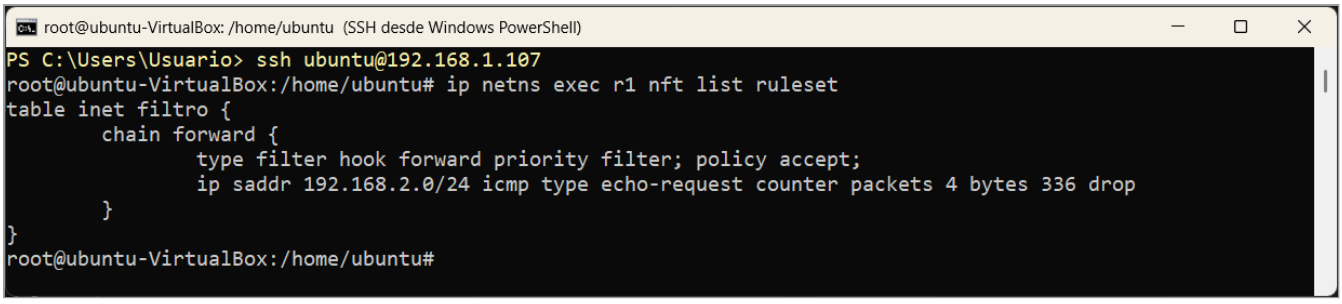
> **Anotar:** pc3 → pc1: 100 % de pérdida · pc1 → pc3: 0 % de pérdida

`capturas\fig12-ubuntu-pings-con-firewall.png`

Captura 41 · Informe 4.6 a)

Contador de la regla después de los pings

nft list ruleset muestra cuántos paquetes descartó la regla.



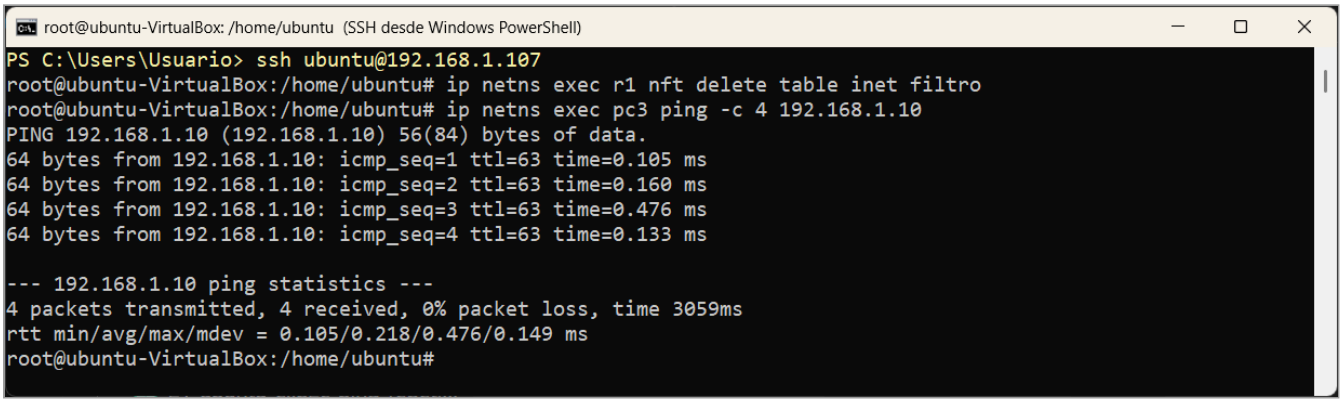
Anotar: Tabla 11 — counter: 4 paquetes, 336 bytes

capturas\25-ubuntu-nft-contador-icmp.png

Captura 42 · Informe 4.6 a)

Quitar la regla y verificar

Se borra la tabla filtro y el ping de pc3 a pc1 vuelve a funcionar.



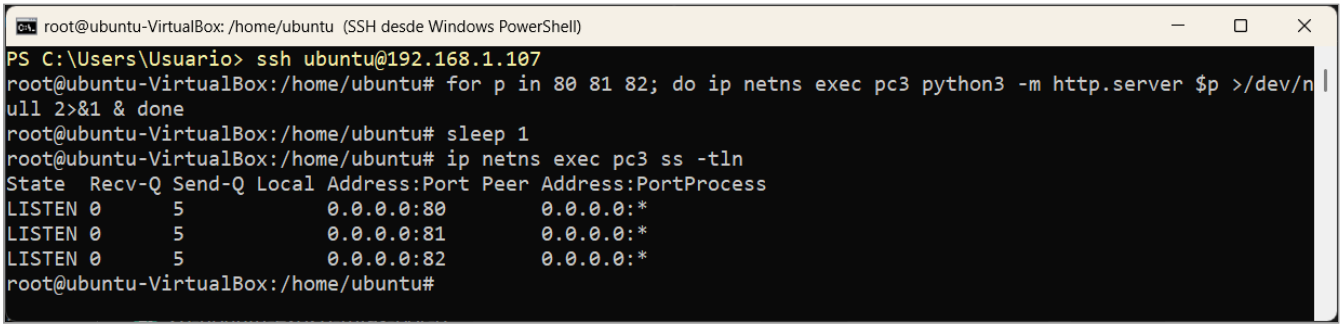
Anotar: 0 % de pérdida

capturas\26-ubuntu-nft-borrar-filtro.png

Captura 43 · Informe 4.6 b)

Servicios de prueba en pc3 (puertos 80, 81 y 82)

Se levantan tres servidores HTTP de Python en pc3, en segundo plano, y ss -tln confirma que escuchan.



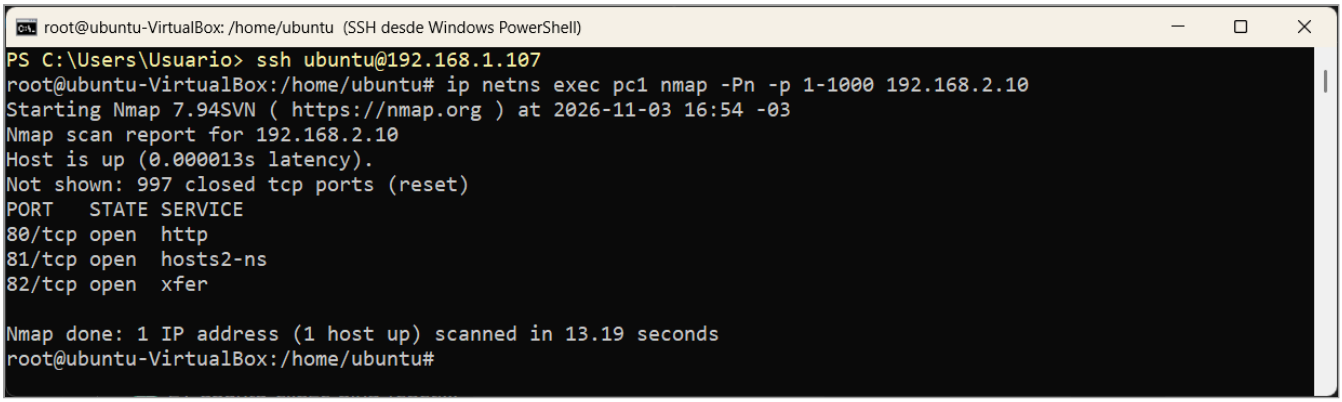
Anotar: LISTEN en 0.0.0.0:80, :81 y :82

capturas\27-ubuntu-http-server-pc3.png

Captura 44 · Informe 4.6 b) — Figura 13

Figura 13: nmap sin firewall

Escaneo de los puertos 1-1000 de pc3 desde pc1, sin reglas.



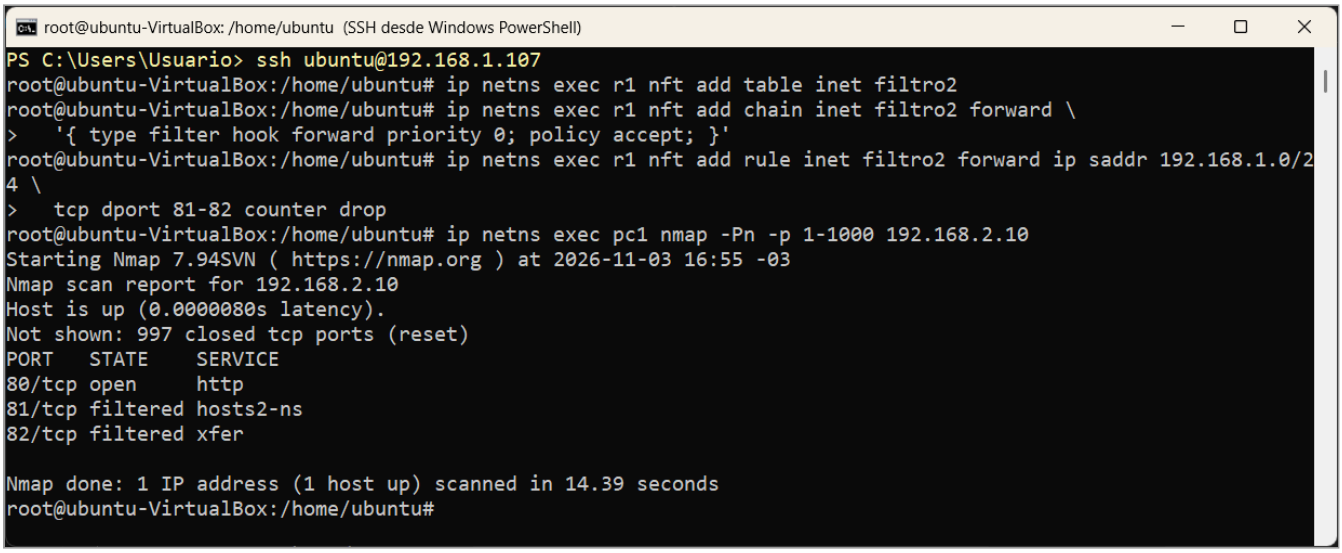
> **Anotar:** Tabla 12 — abiertos: 80, 81, 82 · 997 cerrados · 13,19 s

`capturas\fig13-ubuntu-nmap-sin-firewall.png`

Captura 45 · Informe 4.6 b) — Figura 14

Figura 14: nmap con firewall (puertos 81-82 descartados)

Se crea la tabla filtro2, que descarta el tráfico TCP de la red A hacia los puertos 81-82. Con drop, nmap no recibe respuesta y los marca como filtered.



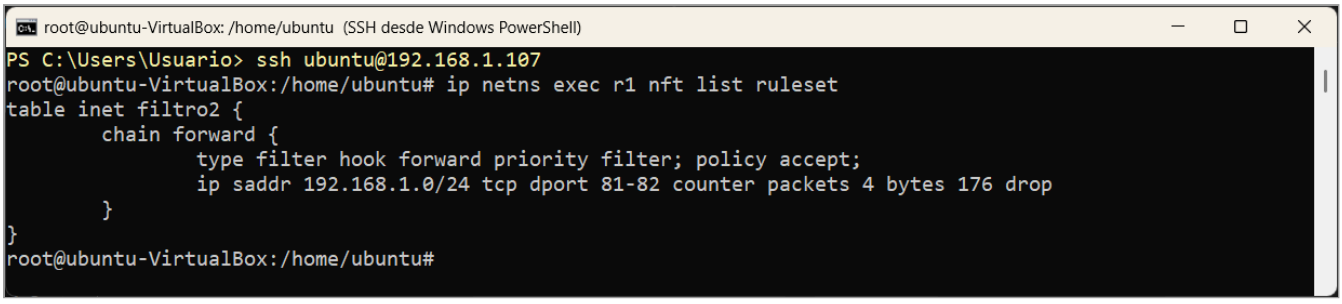
> **Anotar:** Tabla 12 — abiertos: 80 · filtrados: 81, 82 · 14,39 s

`capturas\fig14-ubuntu-nmap-con-firewall.png`

Captura 46 · Informe 4.6 b)

Contador de la regla de puertos

nft list ruleset después del escaneo con firewall.



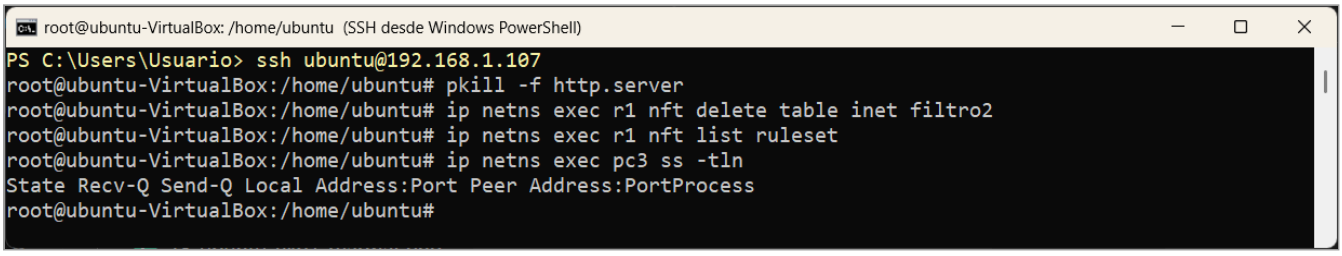
Anotar: counter packets 4 bytes 176 (descartados)

capturas\28-ubuntu-nft-contador-puertos.png

Captura 47 · Informe 4.6 b)

Limpieza de la prueba de puertos

Se detienen los http.server y se borra la tabla filtro2: el ruleset queda vacío y no quedan puertos escuchando en pc3.

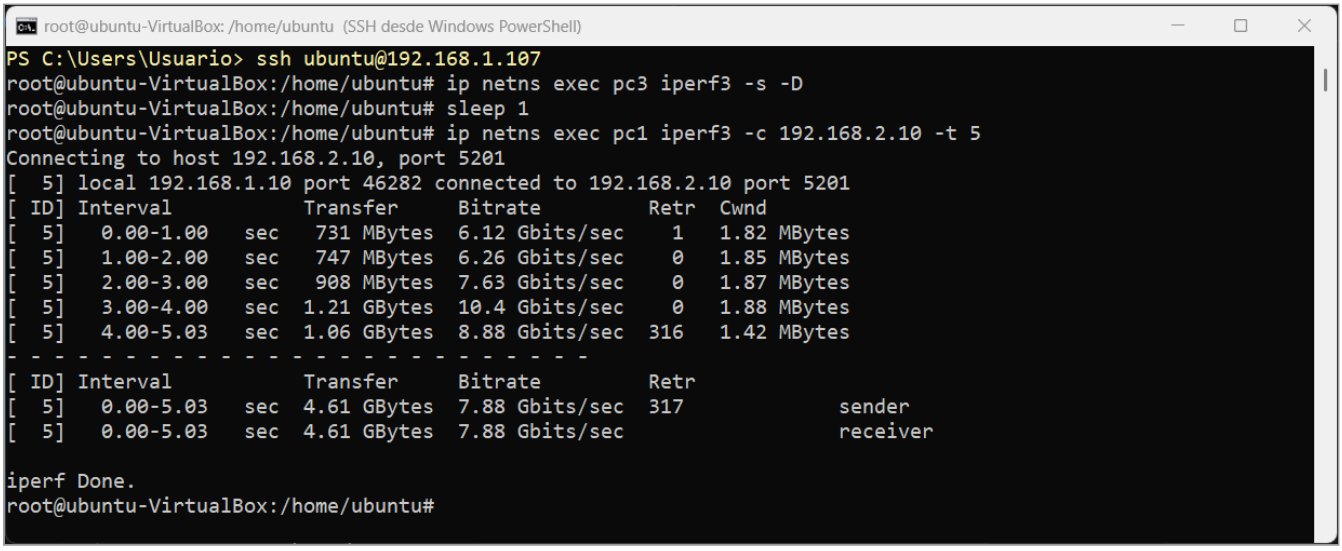


capturas\29-ubuntu-limpiar-exp3b.png

Captura 48 · Informe 4.7 — Experimento 4

iperf3 sin reglas, repetición 1 de 5

Se inicia el servidor iperf3 en pc3 (-s -D) y se mide el rendimiento desde pc1 durante 5 s.

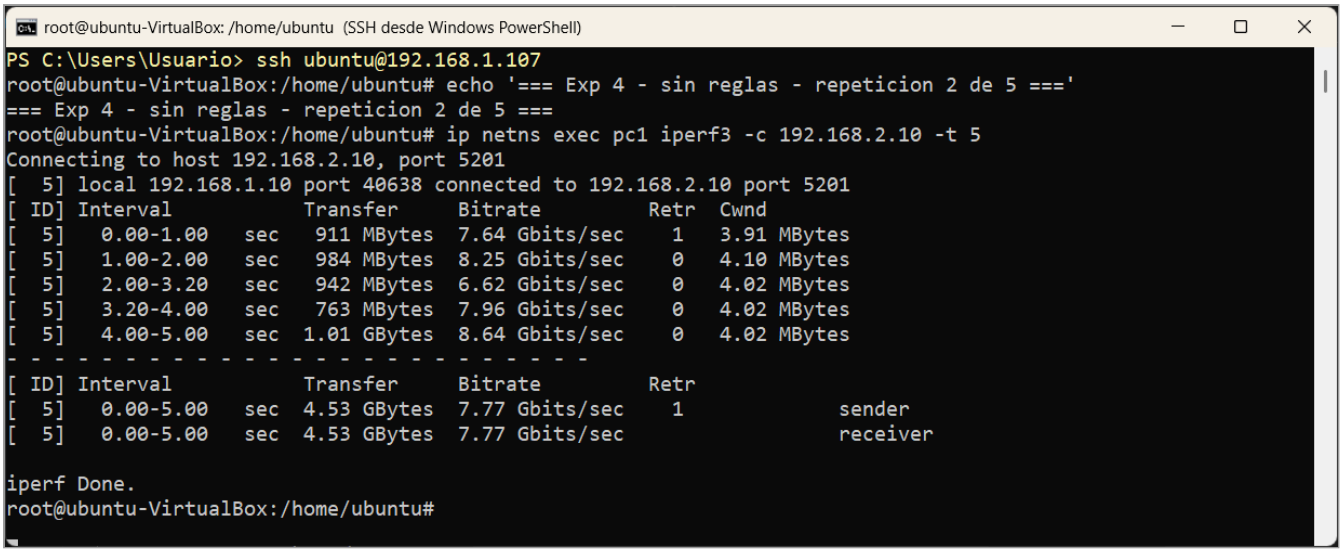


Anotar: receiver: 7,88 Gbits/sec

capturas\30-ubuntu-exp4-0reglas-rep1.png

Captura 49 · Informe 4.7 — Experimento 4

iperf3 sin reglas, repetición 2 de 5

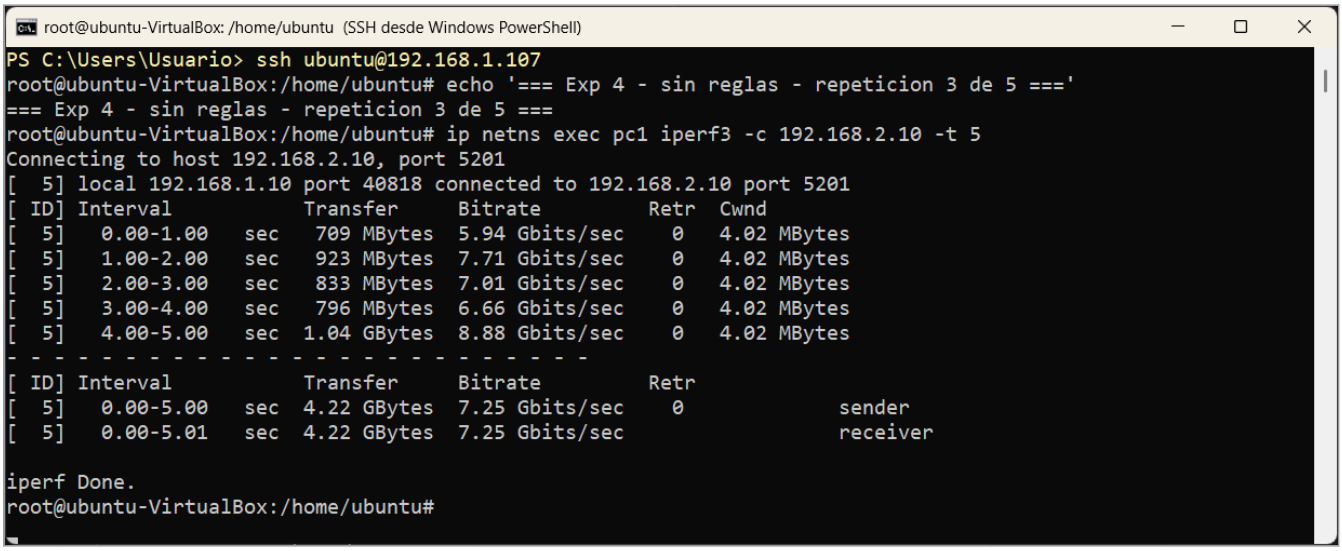


Anotar: receiver: 7,77 Gbits/sec

capturas\30-ubuntu-exp4-0reglas-rep2.png

Captura 50 · Informe 4.7 — Experimento 4

iperf3 sin reglas, repetición 3 de 5

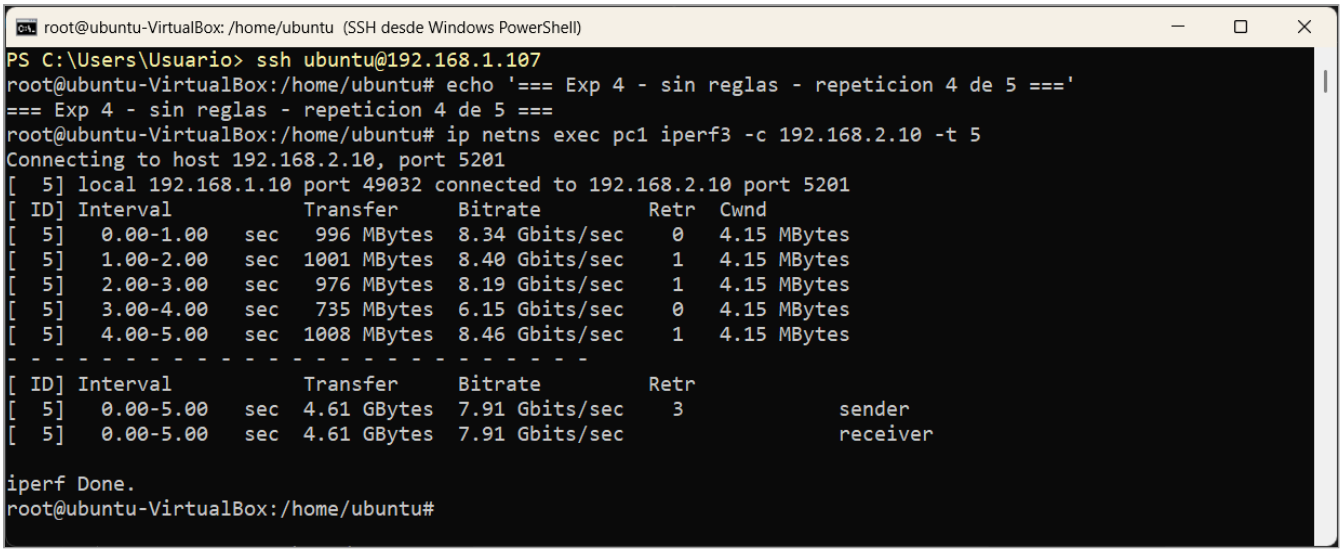


Anotar: receiver: 7,25 Gbits/sec

capturas\30-ubuntu-exp4-0reglas-rep3.png

Captura 51 · Informe 4.7 — Experimento 4

iperf3 sin reglas, repetición 4 de 5

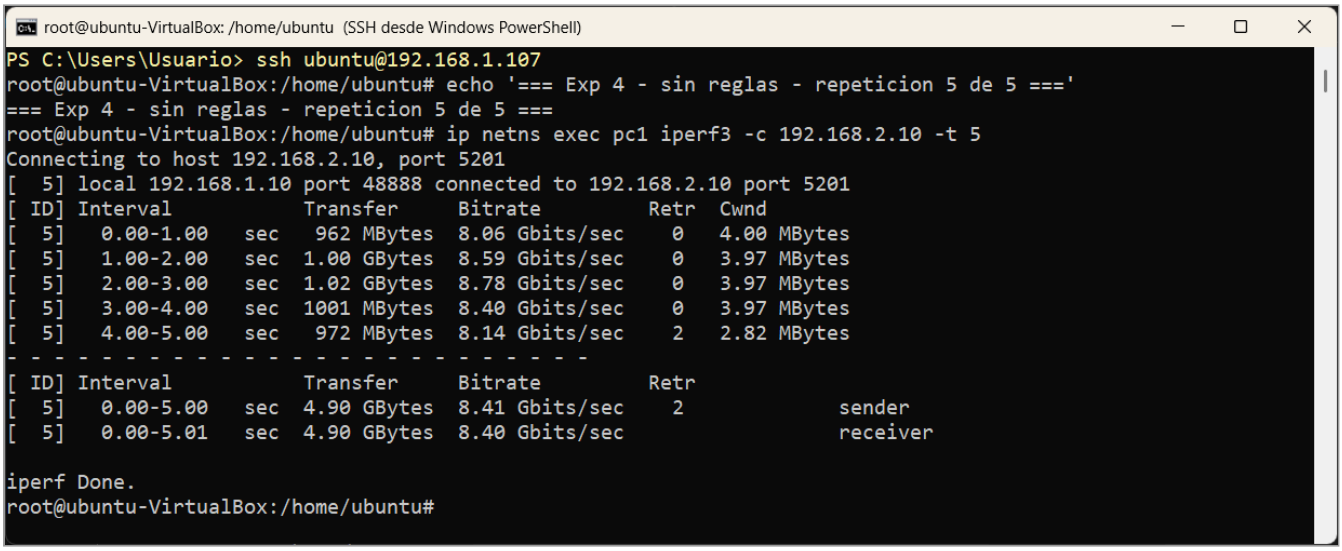


Anotar: receiver: 7,91 Gbits/sec

capturas\30-ubuntu-exp4-0reglas-rep4.png

Captura 52 · Informe 4.7 — Experimento 4

iperf3 sin reglas, repetición 5 de 5



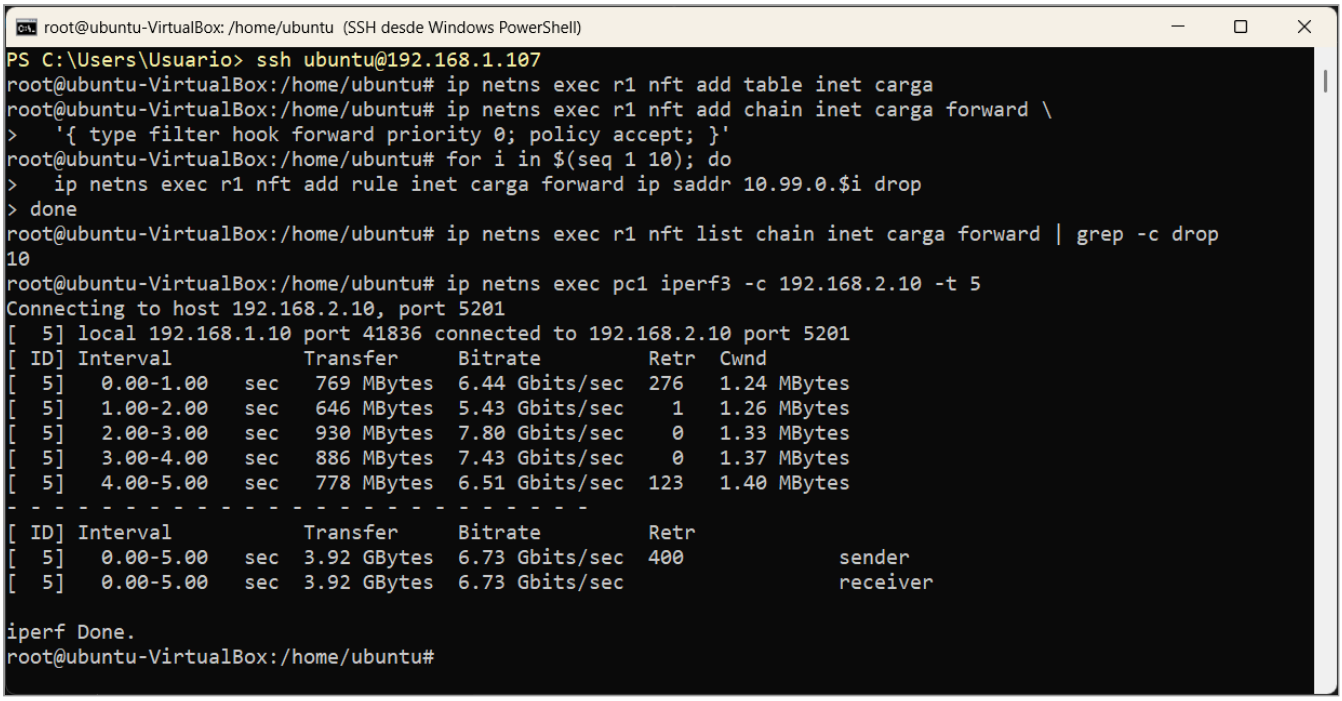
Anotar: receiver: 8,40 Gbits/sec

capturas\30-ubuntu-exp4-0reglas-rep5.png

Captura 53 · Informe 4.7 — Experimento 4

Carga de 10 reglas e iperf3, repetición 1 de 5

Se crea la tabla carga con 10 reglas drop para orígenes 10.99.0.1-10, que no coinciden con el tráfico real, así que cada paquete se compara con todas. grep -c confirma 10 reglas.

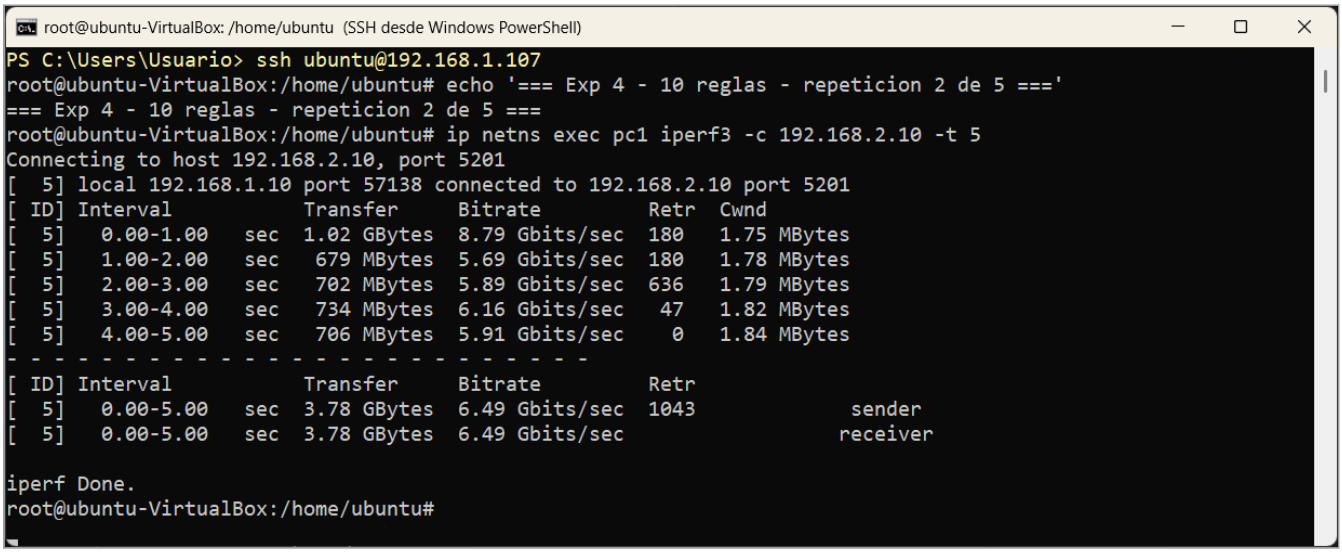


Anotar: receiver: 6,73 Gbits/sec

capturas\31-ubuntu-exp4-10reglas-rep1.png

Captura 54 · Informe 4.7 — Experimento 4

iperf3 con 10 reglas, repetición 2 de 5

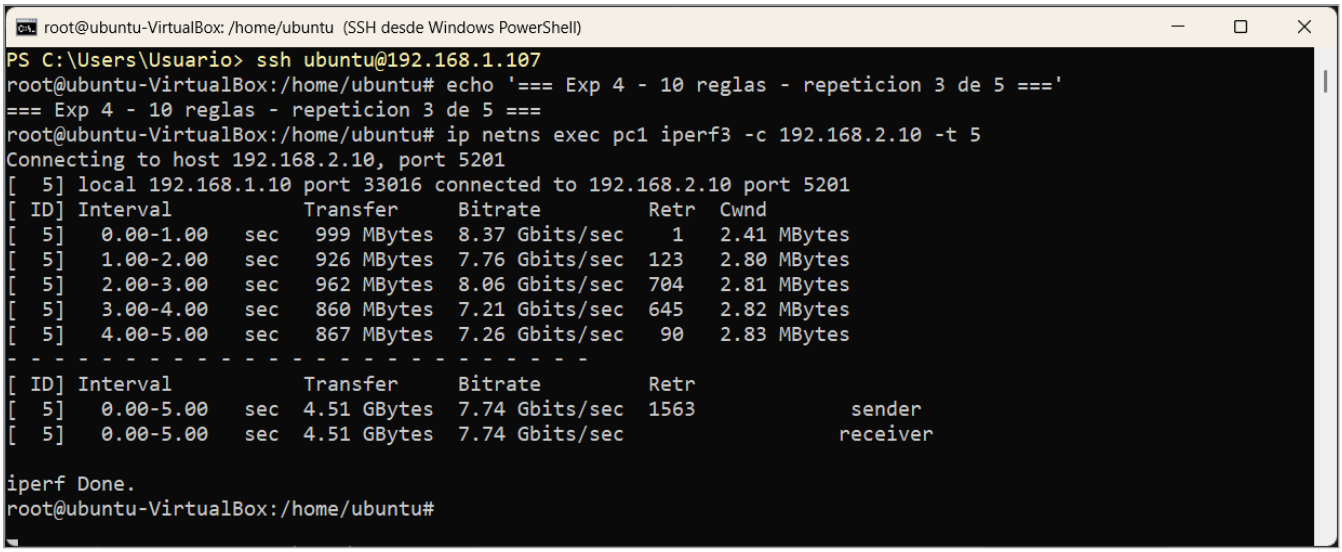


Anotar: receiver: 6,49 Gbits/sec

capturas\31-ubuntu-exp4-10reglas-rep2.png

Captura 55 · Informe 4.7 — Experimento 4

iperf3 con 10 reglas, repetición 3 de 5

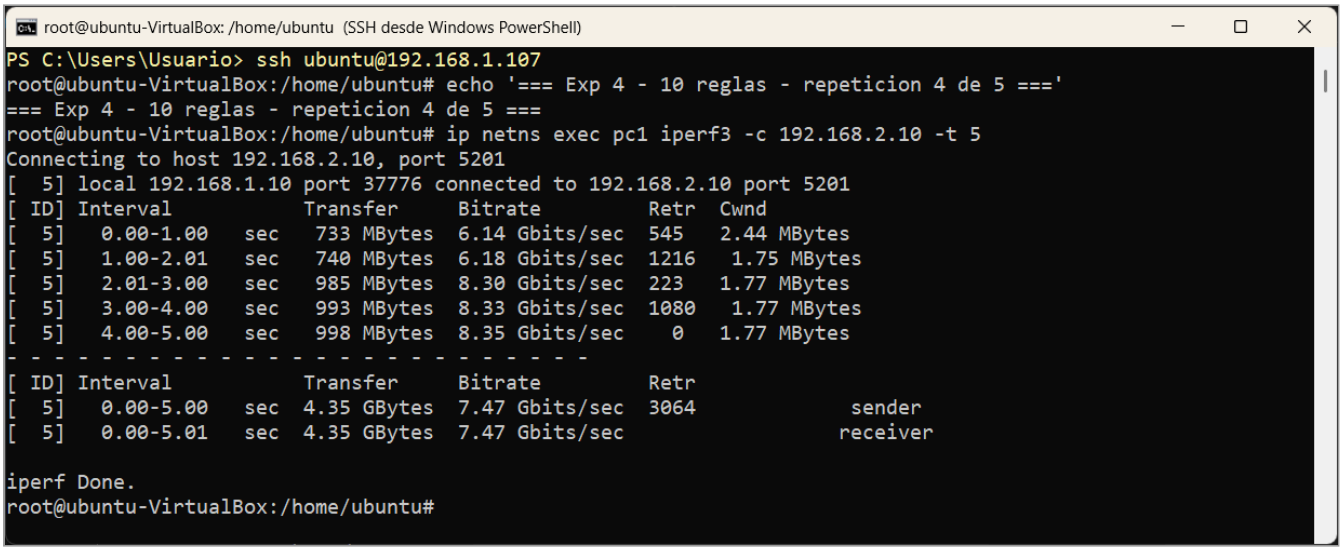


Anotar: receiver: 7,74 Gbits/sec

capturas\31-ubuntu-exp4-10reglas-rep3.png

Captura 56 · Informe 4.7 — Experimento 4

iperf3 con 10 reglas, repetición 4 de 5

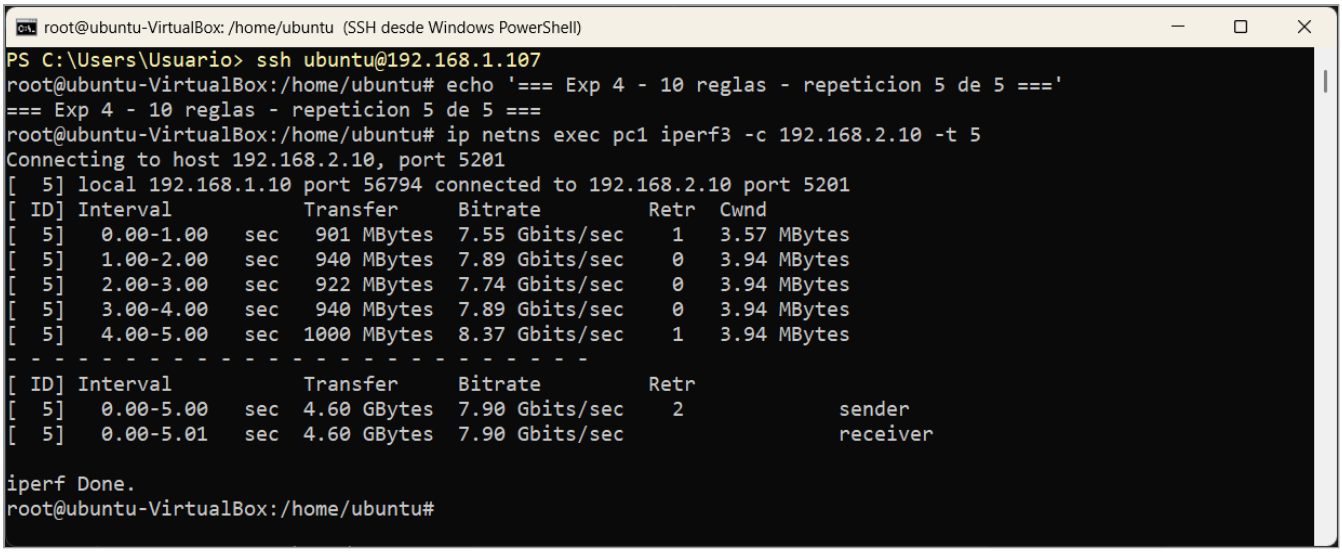


Anotar: receiver: 7,47 Gbits/sec

capturas\31-ubuntu-exp4-10reglas-rep4.png

Captura 57 · Informe 4.7 — Experimento 4

iperf3 con 10 reglas, repetición 5 de 5



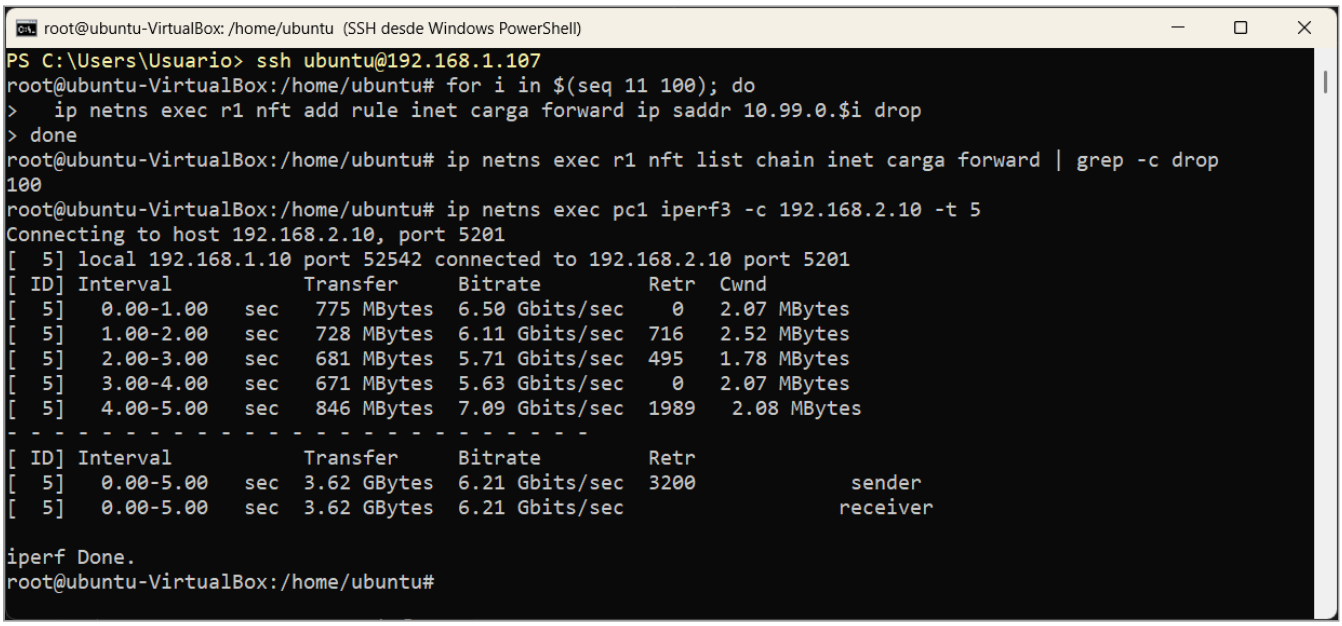
Anotar: receiver: 7,90 Gbits/sec

capturas\31-ubuntu-exp4-10reglas-rep5.png

Captura 58 · Informe 4.7 — Experimento 4

Carga de 90 reglas más (100 en total) e iperf3, repetición 1 de 5

Se agregan las reglas 11 a 100. grep -c confirma 100 reglas en la cadena.

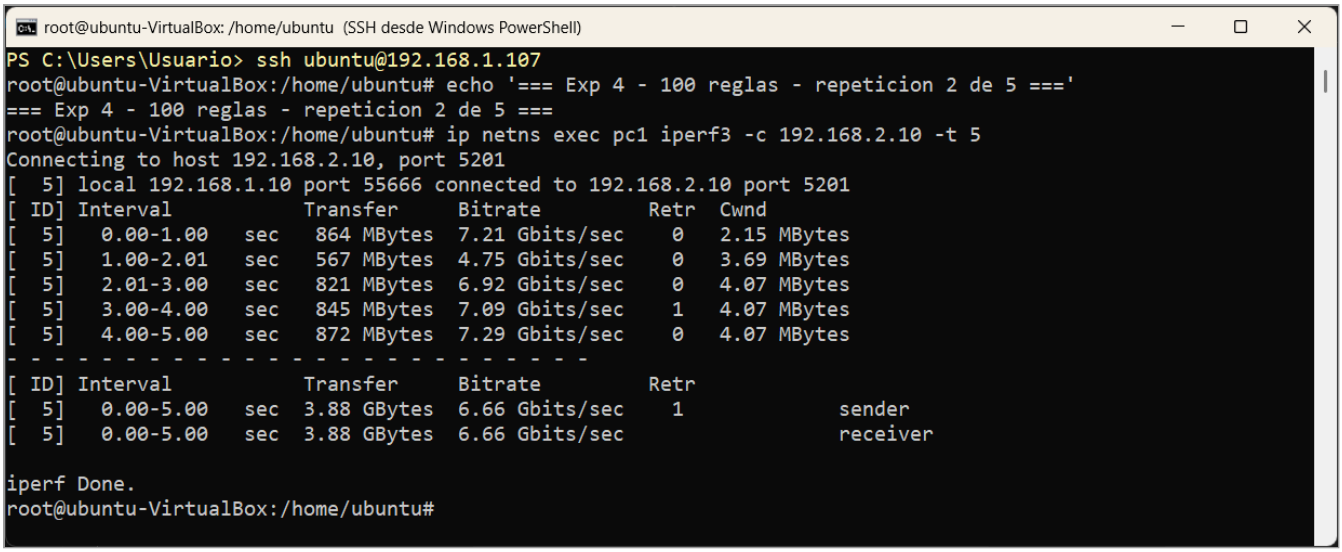


Anotar: receiver: 6,21 Gbits/sec

capturas\32-ubuntu-exp4-100reglas-rep1.png

Captura 59 · Informe 4.7 — Experimento 4

iperf3 con 100 reglas, repetición 2 de 5

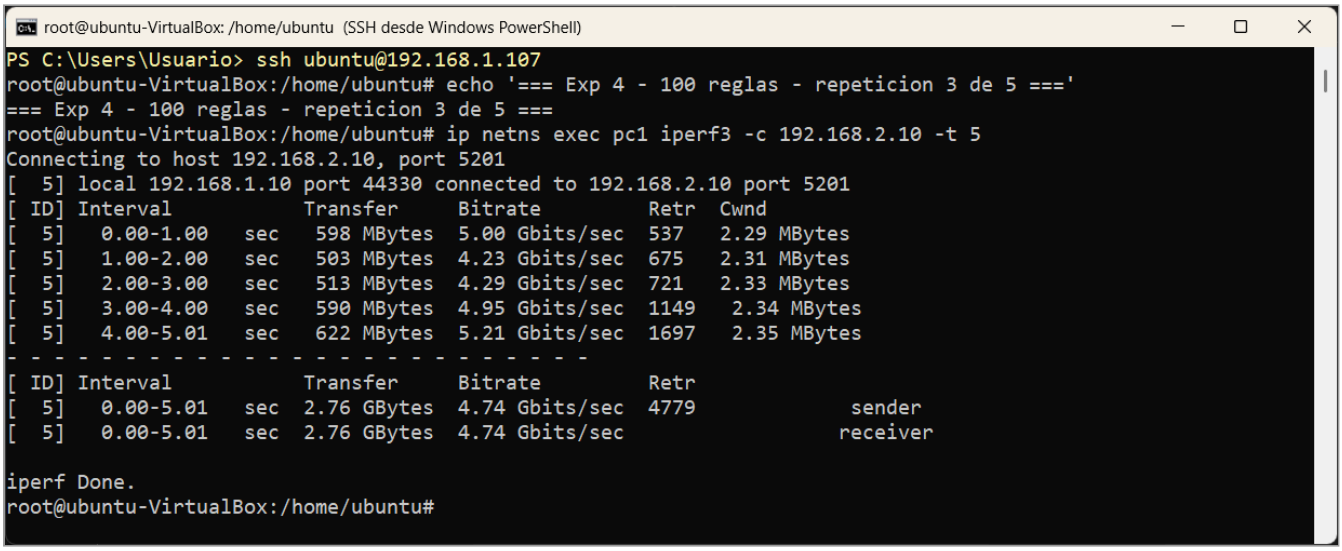


Anotar: receiver: 6,66 Gbits/sec

capturas\32-ubuntu-exp4-100reglas-rep2.png

Captura 60 · Informe 4.7 — Experimento 4

iperf3 con 100 reglas, repetición 3 de 5

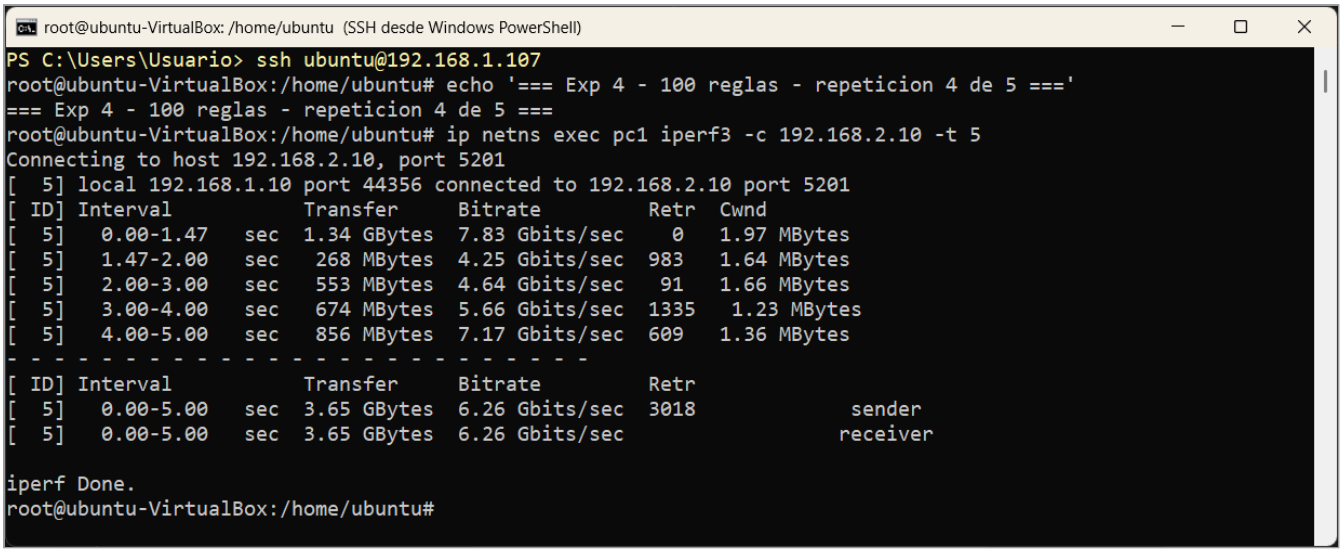


Anotar: receiver: 4,74 Gbits/sec

capturas\32-ubuntu-exp4-100reglas-rep3.png

Captura 61 · Informe 4.7 — Experimento 4

iperf3 con 100 reglas, repetición 4 de 5

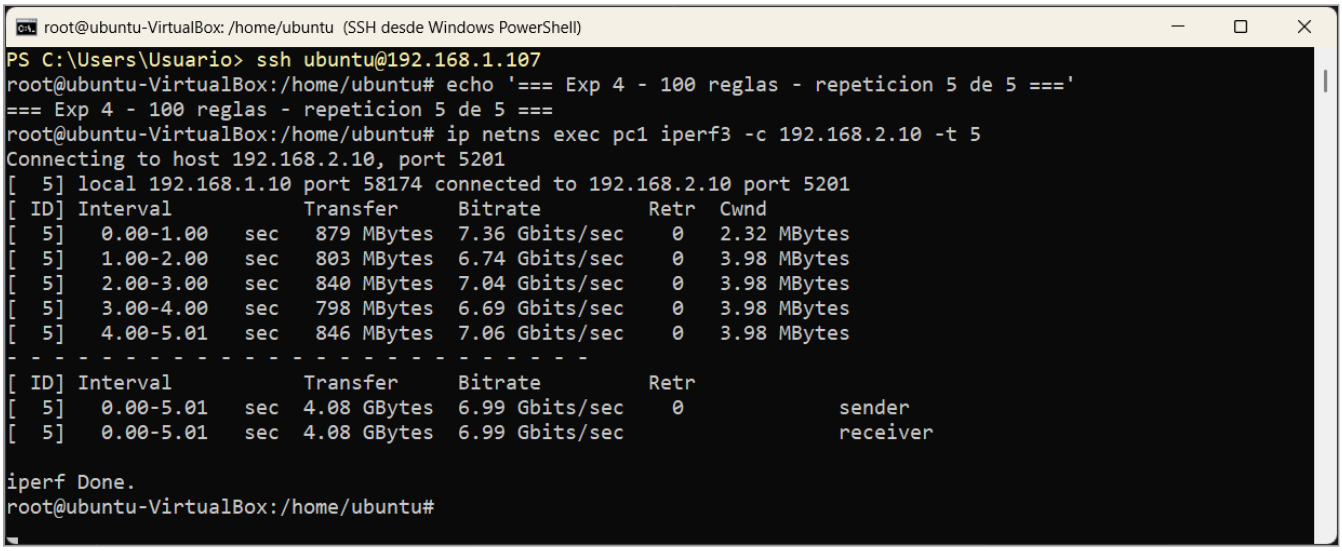


Anotar: receiver: 6,26 Gbits/sec

capturas\32-ubuntu-exp4-100reglas-rep4.png

Captura 62 · Informe 4.7 — Experimento 4

iperf3 con 100 reglas, repetición 5 de 5



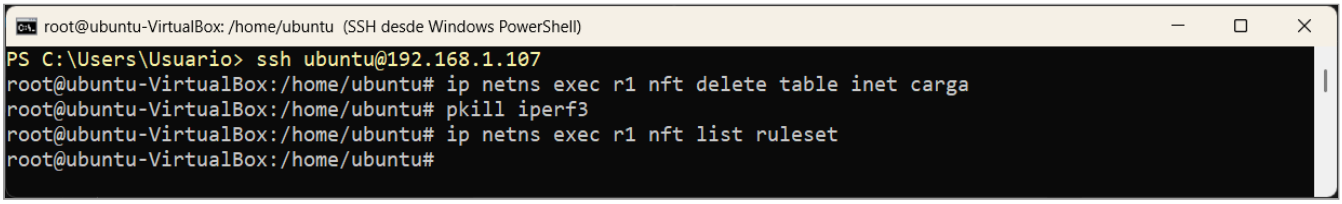
Anotar: receiver: 6,99 Gbits/sec

capturas\32-ubuntu-exp4-100reglas-rep5.png

Captura 63 · Informe 4.7 — Experimento 4

Limpieza del experimento 4

Se borra la tabla carga y se detiene el servidor iperf3. El ruleset de r1 queda vacío.

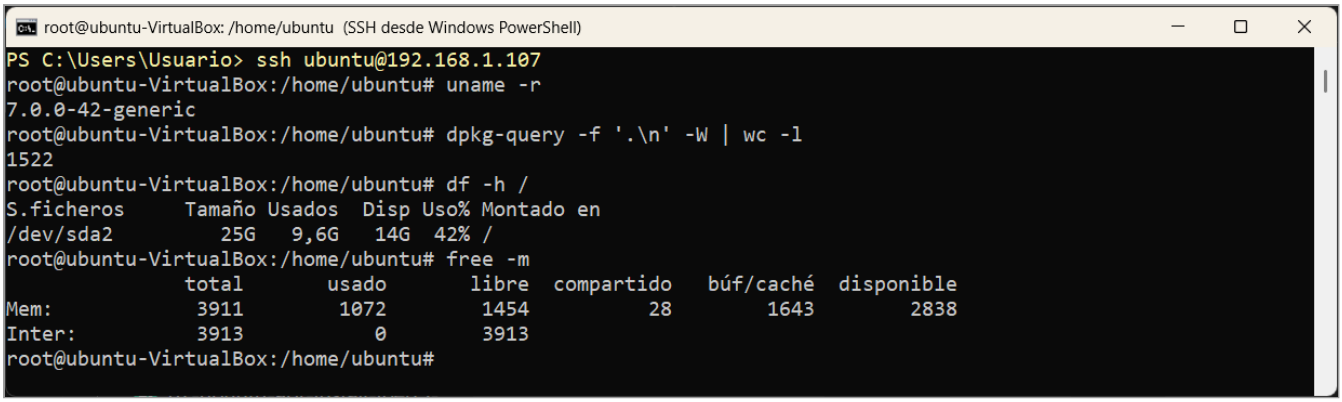


capturas\33-ubuntu-exp4-limpiar.png

Captura 64 · Informe 4.8 — Experimento 5, Parte A

Kernel, paquetes, disco y RAM

Comandos de solo lectura para la Tabla 14.



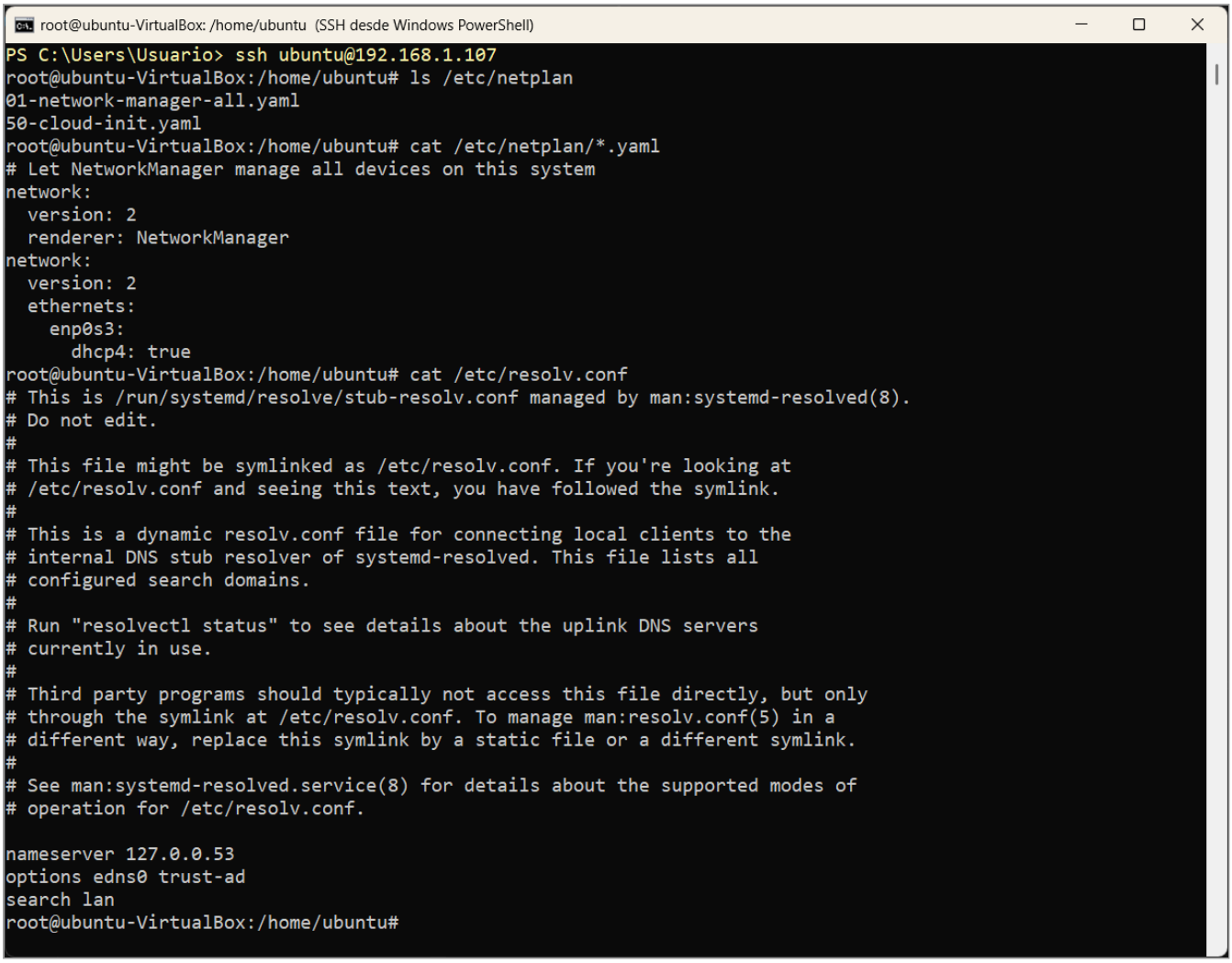
> **Anotar:** Kernel 7.0.0-42-generic · 1522 paquetes · 9,6 GB usados de 25 GB (42 %) · RAM usada 1072 MB de 3911 MB

`capturas\34-ubuntu-exp5a-kernel-paquetes-disco-ram.png`

Captura 65 · Informe 4.8 — Experimento 5, Parte A

Archivos de red (Netplan) y DNS

Aunque es Desktop, Ubuntu tiene /etc/netplan: 01-network-manager-all.yaml le delega toda la red a NetworkManager (renderer: NetworkManager). resolv.conf apunta al stub de systemd-resolved.



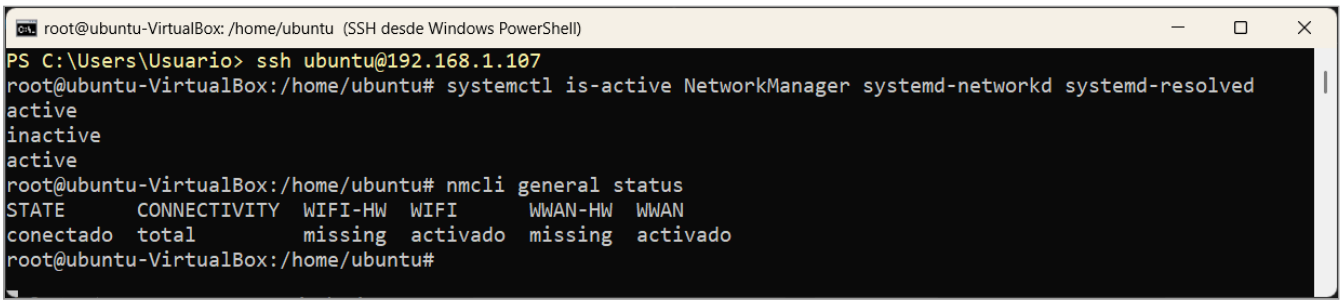
> **Anotar:** Netplan: 01-network-manager-all.yaml y 50-cloud-init.yaml · DNS: nameserver 127.0.0.53 (systemd-resolved), search lan

capturas\35-ubuntu-exp5a-netplan-dns.png

Captura 66 · Informe 4.8 — Experimento 5, Parte A

Servicios de red y estado de NetworkManager

systemctl is-active de los tres gestores posibles y nmcli general status.



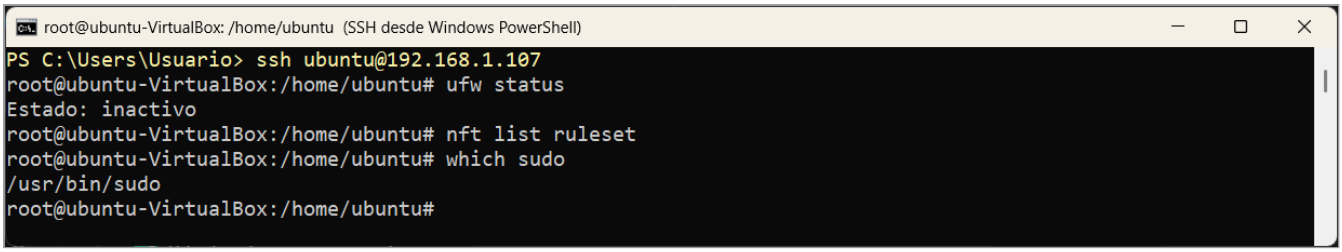
> **Anotar:** NetworkManager: active · systemd-networkd: inactive · systemd-resolved: active · nmcli: conectado, conectividad total

capturas\36-ubuntu-exp5a-servicios-nmcli.png

Captura 67 · Informe 4.8 — Experimento 5, Parte A

Firewall y sudo

ufw viene instalado pero desactivado, el ruleset de nftables del sistema está vacío y sudo está disponible.



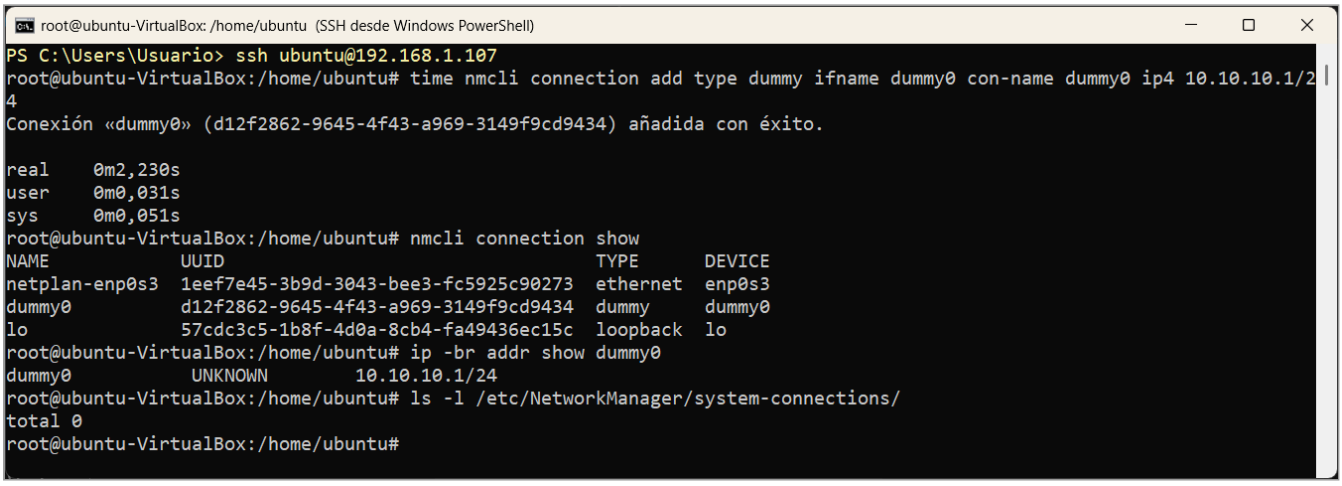
Anotar: ufw: Estado inactivo · nft ruleset: vacío · sudo: /usr/bin/sudo

capturas\37-ubuntu-exp5a-ufw-nft-sudo.png

Captura 68 · Informe 4.8 — Experimento 5, Parte B

Creación de dummy0 con nmcli

Se crea la conexión persistente dummy0 con IP 10.10.10.1/24 y se mide el tiempo. El directorio system-connections queda vacío porque Ubuntu 24.04 guarda las conexiones de NetworkManager como YAML de Netplan.



Anotar: Tiempo del nmcli: real 2,230 s · dummy0 UNKNOWN 10.10.10.1/24

capturas\38-ubuntu-exp5b-nmcli-dummy0.png

Captura 69 · Informe 4.8 — Experimento 5, Parte B

Reinicio de la máquina

Se ejecuta reboot y la sesión SSH se cierra.



capturas\39-ubuntu-exp5b-reboot.png

Captura 70 · Informe 4.8 — Experimento 5, Parte B

VM reiniciando (pantalla de la VM)

Captura de la consola de la VM durante el arranque.

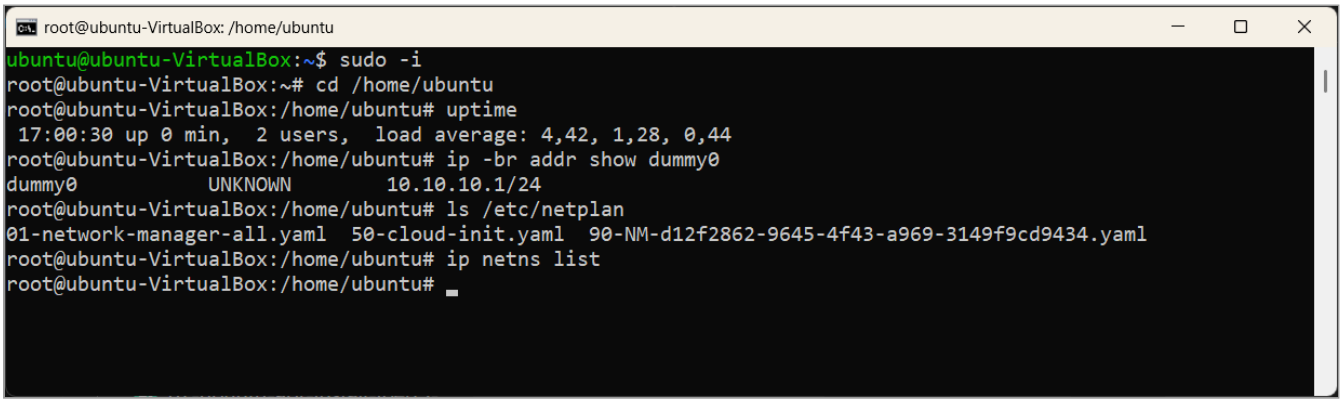


capturas\40-ubuntu-exp5b-vm-reiniciada.png

Captura 71 · Informe 4.8 — Experimento 5, Parte B

Verificación de dummy0 después de reiniciar

Nueva sesión SSH, sudo -i y cd. uptime confirma el reinicio. dummy0 sigue con su IP (es persistente), mientras que los namespaces de la topología desaparecieron.



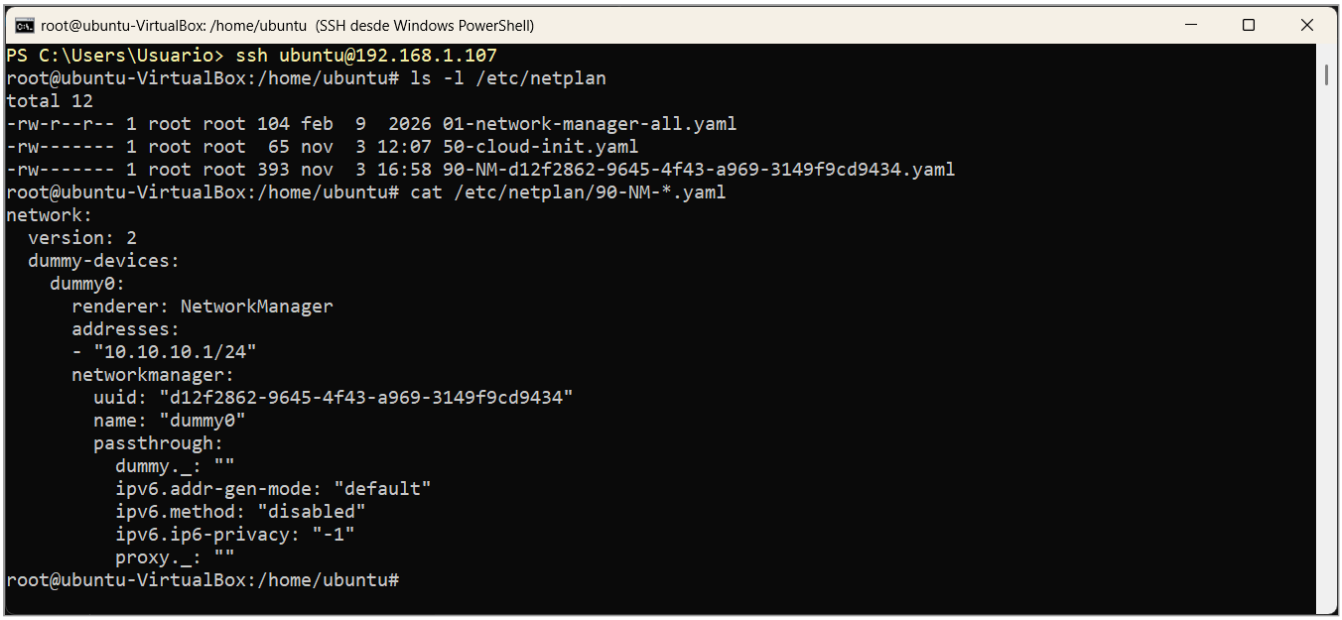
> **Anotar:** IP mantenida tras reiniciar: SÍ (10.10.10.1/24) · ip netns list vacío

capturas\41-ubuntu-exp5b-verificar-tras-reboot.png

Captura 72 · Informe 4.8 — Experimento 5, Parte B

Dónde queda guardada la configuración

NetworkManager guardó la conexión en /etc/netplan/90-NM-<uuid>.yaml (dummy-devices, renderer NetworkManager).



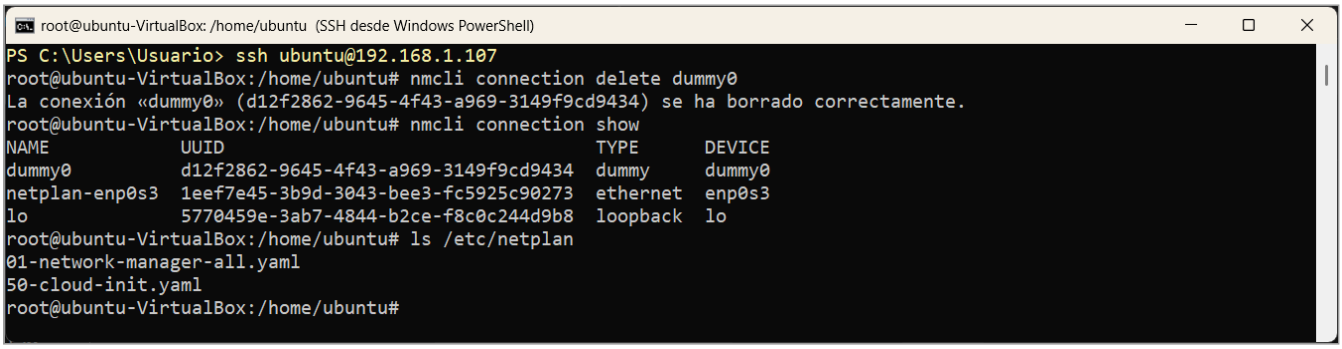
Anotar: Ubicación: /etc/netplan/90-NM-d12f2862-9645-4f43-a969-3149f9cd9434.yaml

`capturas\42-ubuntu-exp5b-donde-se-guarda.png`

Captura 73 · Informe 4.8 — Experimento 5, Parte B

Borrado de dummy0 (dejar la máquina como estaba)

nmcli connection delete dummy0 borra la conexión y su YAML de /etc/netplan. En el nmcli connection show inmediato todavía aparece listada mientras NetworkManager recarga la configuración.



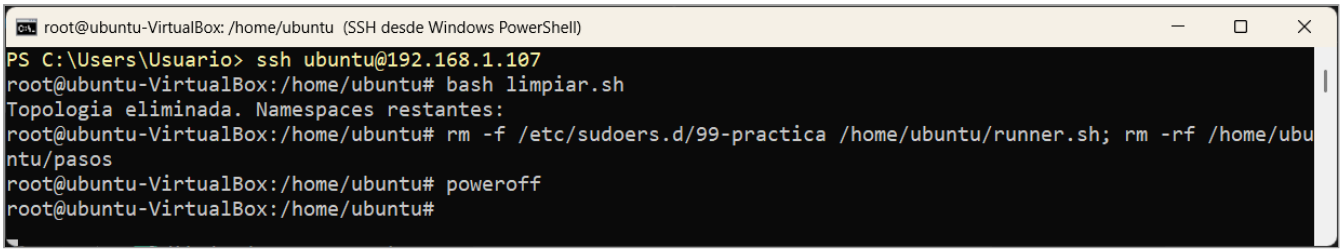
Anotar: Archivo 90-NM-*.yaml eliminado

`capturas\43-ubuntu-exp5b-borrar-dummy0.png`

Captura 74 · 8. Al terminar

limpiar.sh y poweroff

Se borra la topología, se quitan los archivos temporales de la automatización (sudoers temporal, runner.sh) y se apaga la máquina.



> **Anotar:** VM apagada (VMState=poweroff)

`capturas\44-ubuntu-final-limpiar-poweroff.png`